VIETNAM NATIONAL UNIVERSITY HCMC

INTERNATIONAL UNIVERSITY

School of Computer Science and Engineering

Principle of Database Management

FINAL REPORT

HOTEL MANAGEMENT SYSTEM

STT	Student name	Student ID
1	Trần Thanh Phương	ITDSIU21062
2	Trần Phúc Khang	ITDSIU21089
3	Nguyễn Nhật Tiến	ITDSIU23023
4	Nguyễn Hoàng Bảo	ITDSIU23004

Table 1: Member's name and contribution

Course: Principle of Database Management

Academic advisor: Assoc. Prof. Nguyen Thi Thuy Loan

VIETNAM NATIONAL UNIVERSITY HCMC

INTERNATIONAL UNIVERSITY

FINAL REPORT

STUDENT RESEARCH PROJECT

Hotel Management System

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION 7

1.1. Abstraction: 7

1.2. System Overview: 7

1.3. Goal: 7

CHAPTER 2. TASK TIMELINE & DIVISION 8

2.1. Contribution 8

2.2. Project Timeline & Task Division 8

CHAPTER 3. PROJECT ANALYSIS 10

3.1. Requirement analysis 10

3.2. Approach analysis 11

Reviewed materials 11

Research Approach 11

3.3. System analysis 12

3.3.1 Database Design 12

3.3.2 Database Normalization 13

3.4. Database and Table Creation 14

3.5. Database Data Insertion 17

3.6. Database Query 21

3.6.1. Query Analysis 21

3.6.2. Web Interface 22

3.6.3. Dashboard 23

3.6.4. Application Interface 23

CHAPTER 4. CONNECTION BETWEEN SQL AND JAVA 26

CHAPTER 5. DEMO PROJECT 28

5.1. The user experience: 28

5.1.1. Overview: 28

5.1.2. Register user: 28

5.1.3. Booking: 29

5.1.4. Dashboard: 31

5.2. The hotel management system application: 33

5.2.1. Login: 33

5.2.2. Dashboard: 34

5.2.3 Manage employee: 35

5.2.4. Manage room: 38

5.2.5. Manage booking: 41

5.2.6. Payment 45

CHAPTER 6. CONCLUSION AND RECOMMENDATION 47

6.1 Achieved goals 47

6.2. Future Work 47

6.3. Concluding thought 47

REFERENCES 48

LIST OF FIGURES

***Figure 1**: Agile methodology* 11

Figure 2: *Primary Crow's Foot Format E.R.D. Diagram* 12

***Figure 3:** Schema design generated by SQL Server Management Studio* 13

***Figure 4:** Connection Database via JDBC* 27

***Figure 5:** Database Connection via PHP* 27

***Figure 6:** The index page* 28

***Figure 7:** Login site* 29

***Figure 8:** Register account* 29

***Figure 9:** Scroll down to the list of room type to book the reservation* 30

***Figure 10:** Fill the information for reservation* 30

***Figure* 11*:** Reservation successful* 31

***Figure 12:** Failure reservation due to room type unavailable* 31

***Figure 13:** The main index site* 32

Figure 14*: Dashboard site* 32

***Figure 15:** Cancel reservation successfully* 33

***Figure 16:** Login interface* 34

***Figure 17:** Dashboard interface* 35

Figure 18: *Employee Management interface* 35

***Figure 19:** Fill the employee information* 36

***Figure 20:** Success panel show* 36

***Figure 21:** Choose employee to modify* 37

***Figure 22:** Modify Information* 37

***Figure 23:** Success changing panel* 38

***Figure 24:** Value after modification* 38

***Figure 25:** Remove employee* 38

***Figure 26:** Room interface* 39

***Figure 27:** Add room type information* 39

***Figure 28:** Adding success* 39

***Figure 29:** Room table record a new room* 40

Figure 30: *Set room status successfully* 40

Figure 31: *Update status* 40

Figure 32: *Remove room announcement* 41

Figure 33: *Booking interface* 41

Figure 34: *Filling reservation information* 42

Figure 35: *Reservation successfully* 42

Figure 36: *Save booking information into new table* 43

Figure 37: *Modify information* 43

Figure 38: *Modify successfully* 44

Figure 39: *System receive booking information from website* 44

Figure 40: *Update room status* 45

Figure 41: *Update succesfully* 45

Figure 42: *Payment interface* 45

Figure 43: *Update payment details* 46

Figure 44: *Update payment successfully* 46

Figure 45: *Payment information after create bill* 46

LIST OF TABLES

Table 1: Member's name and contribution 1

Table 2: Individual responsibility 8

Table 3: Sprint planning and task division for individuals and teams 10

Table 4: An overview of all database entities and their attributes 14

CHAPTER 1. INTRODUCTION

1.1. Abstraction:

The booming tourism industry demands that hotels innovate and streamline operations to stay competitive. This Hotel Management System (HMS) project provides a four-step approach to building a robust system that enhances guest experiences and optimizes hotel management. These steps include:

1. **Mapping the Landscape:** Using entity-relationship diagrams (ERDs) to model key elements like rooms, guests, staff, and services for efficient data storage.
2. **Structuring the Database:** Employing database normalization to minimize data redundancy and ensure data integrity, leading to faster access and fewer errors.
3. **Optimizing Queries:** Developing efficient SQL queries for quick and accurate data retrieval, enabling staff to efficiently access reservations, guest details, and other essential information.
4. **Integrating Data Sources:** Unifying various data sources, such as online booking platforms and inventory systems, for a holistic view of hotel operations and data-driven decision-making.

This HMS project focuses on both user experience and functionality. Guests can easily search and book rooms, while staff can manage reservations, track availability, and streamline communication. Secure payment processing and user-friendly interfaces for both staff and guests ensure a seamless experience.

This project offers students and professionals practical experience in managing real-world hotel data, building scalable and efficient systems, and implementing best practices for data integrity, security, and performance.

1.2. System Overview:

Hotel Management Systems (HMS) are crucial in today's increasingly digital hospitality industry. Similar to online marketplaces, HMS platforms connect hotels and guests conveniently. Guests enjoy seamless booking, reservation management, and easy access to information, while hotels benefit from streamlined operations and improved communication. This digital transformation allows hotels to enhance guest experiences and compete effectively. By adopting HMS, hotels can achieve greater efficiency, increased guest satisfaction, and a stronger competitive edge.

1.3. Goal:

- **Data Integrity:** Maintain accurate and consistent guest information by eliminating errors and inconsistencies. Establish a single, reliable source for all guest data.

- **Guest Communication:** Efficiently store and manage guest communication data, such as inquiries and complaints, and enable smooth, timely interactions across multiple channels.

- **Order Management:** Effectively manage reservations, track room availability, and streamline booking processes to provide guests with aseamless and efficient booking experience.

CHAPTER 2. TASK TIMELINE & DIVISION

2.1. Contribution

Table 2: Individual responsibility

Name	Responsibility
Bảo	UI designer, Database developer (main)
Tiến	Database developer (main)
Phương	Database(vice), UI developer
Khang	Database(vice), UI developer

2.2. Project Timeline & Task Division

Stage	Task	Member
PLANNING	Research information and experience about famous hotel management platform	All
	Collect sample data for analyzing	All
	Research for reference and technical documentation	All
	Unanimously decide on which communication, workflow, and tools to use	All
	Determine Scopes, objectives, and goals for project	All
	Analyze proper for approach and methodology for project development	All
	Specify requirements for the project	All

	Create timeline table and division task	Khang
	Determine proper technology stack and database management system	All
	Determine features and their priorities	All
	Proposal report	Khang, Phương
	Phase 1 review and retrospective	All
CONCEPTUAL DESIGN	Identify use case and actors of the system	All
	Specify types of information that are essential	All
	Design user interface for core functionalities	Tiến, Bảo
	Design the relational models	All
	Design E.R diagram and class diagram	Khang, Tiến
	Midterm report	All
	Phase 2 review and retrospective	All
IMPLEMENTING	Database creation and setup	All
	Develop web user interface and client-side functionality	Khang, Phương
	Work on new registration	Phương
	Work on user login/logout and authentication	Phương
	Build dashboard website for user	Khang
	Develop login credential of application	Khang
	Develop dashboard implement for manager including showing list of booking, employees, customers, room/room type, and payment	Bảo

	Develop employee management that showing the number of employees and roles of each employee	Tiến
	Develop room management that showing list of rooms attach to type of rooms and price per room	Tiến
	Develop booking management that display all pending reservation from user and add the option to create/check for guest coming to hotel	Bảo
	Develop payment management that check which booking have paid or not	Bảo
	Phase 3 review and retrospective	All
TESTING	Code review and refactoring	All
	Bug detection and fixing	Tiến, Bảo
	Design and implement test case	Phương
	Phase 4 review and retrospective	All
PRESENTATION	Final report	All
	Slides report	Khang, Phương
	Listing requested information and finishing report	All

Table 3: Sprint planning and task division for individuals and teams

CHAPTER 3. PROJECT ANALYSIS

3.1. Requirement analysis

The Hotel Management System provides with the following functionalities:

- **Make a reservation:** Book a room for a customer by gathering all necessary details and verifying room availability in the hotel.
- **Room Status:** Users can view the current status of rooms, including how many are vacant and occupied.
- **Check in and use of facilities:** Once a reservation is confirmed, customers can check in and access the hotel's facilities such as the restaurant, laundry service, tea service, and more.

- **Check out: During check-out, the staff calculates and presents the customer's final bill for payment.**

3.2. Approach analysis

Reviewed materials

+ HTML/CSS/JavaScript/PHP/MySQL/Java (JRE 8/JDK 22.0 above)
+ IDE: Visual Studio/ NetBeans
+ Intermediate SQL Server language structure ("SQL Tutorial," n.d.)

Research Approach

- *Methodology*

  Agile methodology is a project management approach that prioritizes fast delivery, adaptability to changing requirements, and strong collaboration among team members. It promotes an iterative process with regular reviews and updates to ensure the final product aligns with the customer's expectations.

- *Stages of the Agile software development lifecycle*

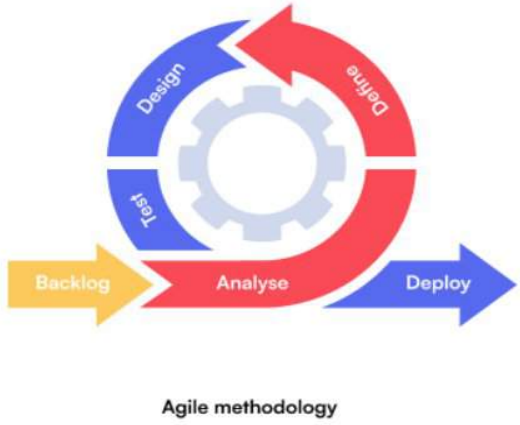


***Figure 1**: Agile methodology*

***Figure** 1: Agile methodology*

3.3. System analysis

3.3.1 Database Design

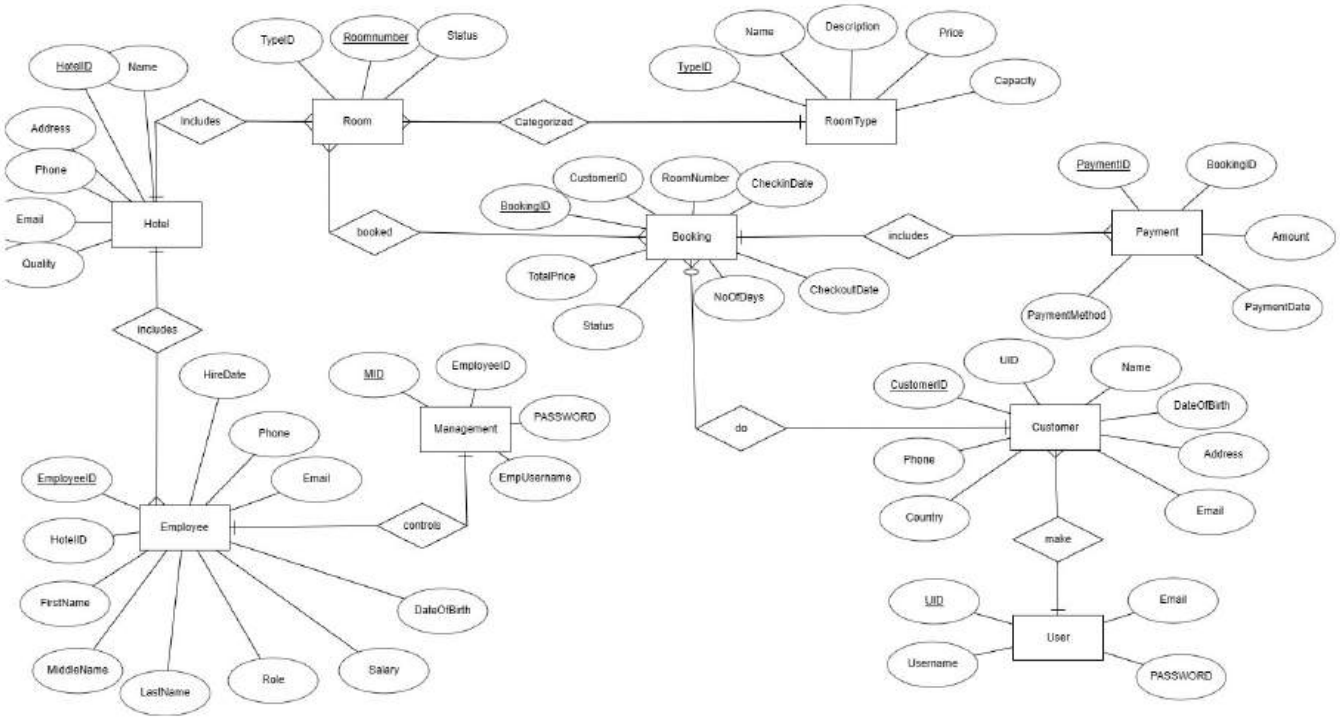


Figure 2: *Primary Crow's Foot Format E.R.D. Diagram*

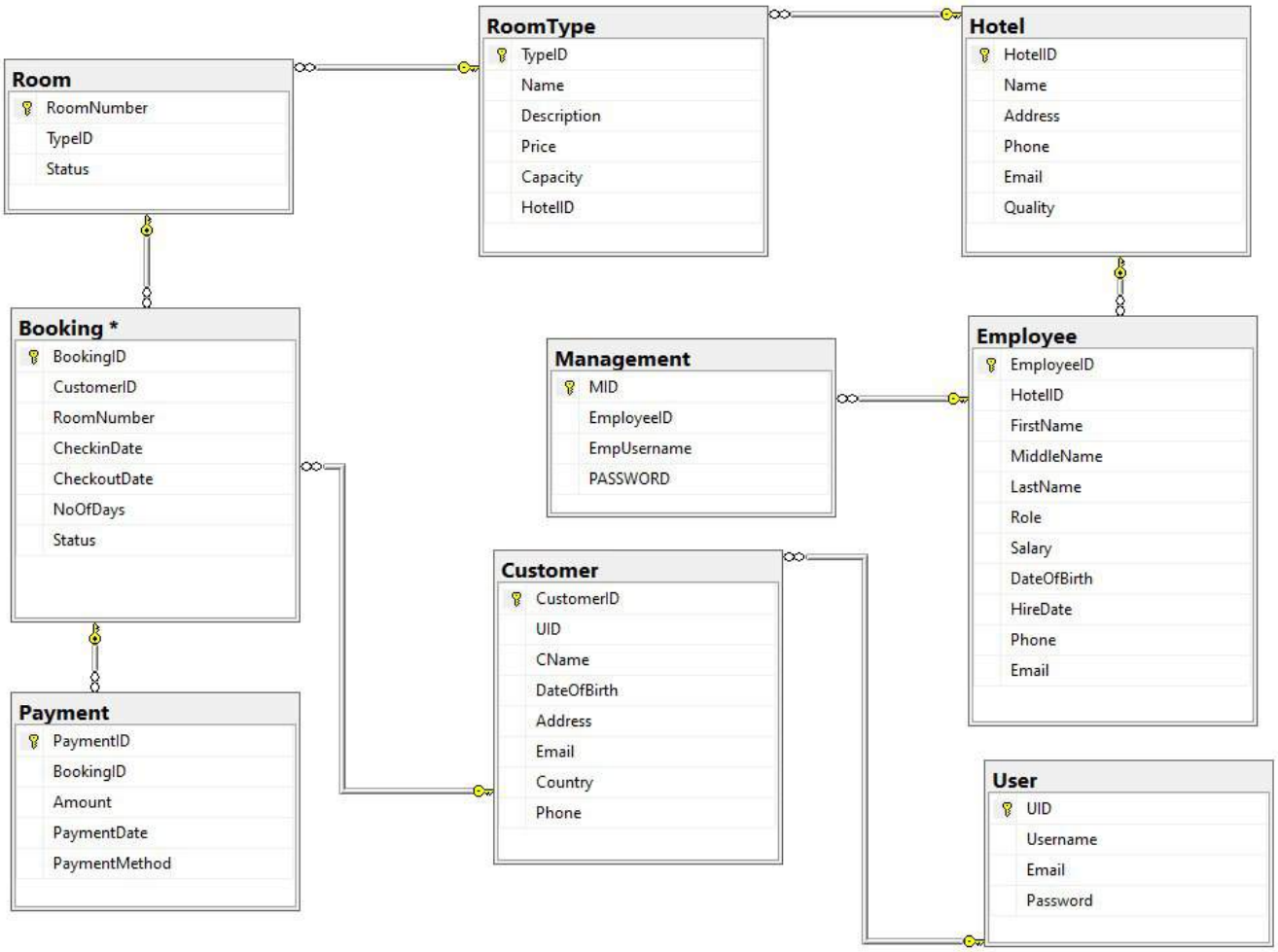


Figure 3: *Schema design generated by SQL Server Management Studio*

3.3.2 Database Normalization

Main purpose

Data normalization involves organizing a database's structure to remove redundancy, eliminate unnecessary or inconsistent information, and maintain data consistency and integrity. Here's how we implement data normalization in our Hotel Management Database System:

Normalization process

Step 1: 1st Normal Form (1NF)

- A single cell must not hold more than one value (atomicity).
- There must be a primary key for identification.
- No duplicated rows or columns.

Step 2: 2nd Normal Form (2NF): A table is in the second normal form if it is in the first normal form and every non-key attribute is fully functionally dependent on the primary key

Step 3: 3rd Normal Form (3NF): A table is in the third normal form if it is in the second normal form and there are no transitive functional dependencies.

Step 4: Boyce-Codd Normal Form (BCNF): Each determinant in a table should be a candidate key.

Conclusion: after following these steps above the database is 3NF.

3.4. Database and Table Creation

This part discusses the database creation, schemas, and tables following the E.R.D, the diagram in SQL Server Database:

Database Creation:

```
CREATE DATABASE HMS;

USE HMS;
```

Tables Analysis:

The current version of the Entity-Relationship Diagram (E.R.D.) has the following entities:

Table 4: An overview of all database entities and their attributes

Table	Attributes	Explain
Hotel	**HotelID** INT PRIMARY KEY default 10, **Name** VARCHAR(50) default 'HCMIU', **Address** VARCHAR(100) default 'Quarter 6, Linh Trung ward, Thu Duc, HCMC', **Phone** VARCHAR(15) default '(028) 37244270', **Email** VARCHAR(100) default 'info@hcmiu.edu.vn', **Quality** INT default 5	The information of the hotel about name, address, phone, email, and hotel star rating(quality) for contact.
RoomType	**TypeID** INT PRIMARY KEY not null auto_increment, **Name** VARCHAR(50), **Description** VARCHAR(255), **Price** DECIMAL(10, 2), **Capacity** INT,	Store type of room with details such as name, description, price, capacity, and hotel which belong to.

	HotelID INT default 10, FOREIGN KEY (HotelID) references Hotel(HotelID)	
Room	**RoomNumber** INT PRIMARY KEY not null auto_increment, **TypeID** INT, **Status** VARCHAR(20), FOREIGN KEY (TypeID) REFERENCES RoomType(TypeID)	Showing what room is available or not and type of this room
Employee	**EmployeeID** INT not null PRIMARY KEY auto_increment, **HotelID** INT default 10, **FirstName** VARCHAR(25), **MiddleName** VARCHAR(25), **LastName** VARCHAR(25) NOT NULL, **Role** VARCHAR(25) NOT NULL, **Salary** DECIMAL(10, 2), **DateOfBirth** DATE, **HireDate** DATE, **Phone** VARCHAR(15), **Email** VARCHAR(255), FOREIGN KEY (HotelID) REFERENCES Hotel(HotelID)	Store information of individual employees who work in the hotel like their employee ID, their name, role, salary, date of birth, hire date, phone, email.
Management	**MID** INT PRIMARY KEY not null auto_increment, **EmployeeID** INT NOT NULL, **EmpUsername** VARCHAR(10), **PASSWORD** VARCHAR(64) NOT NULL, FOREIGN KEY (EmployeeID)	Provide manager credential to manage almost function like manage employee, booking, customer, payment in application.

	REFERENCES Employee(EmployeeID)	
User	**UID** INT PRIMARY KEY not null auto_increment, **Username** VARCHAR(16) NOT NULL, **Email** VARCHAR(32) NOT NULL, **Password** VARCHAR(32) NOT NULL	User credential who wants to book room online.
Customer	**CustomerID** INT PRIMARY KEY not null auto_increment, **UID** INT NOT NULL, **CName** VARCHAR(50) NOT NULL, **DateOfBirth** DATE, **Address** VARCHAR(70), **Email** VARCHAR(70), **Country** VARCHAR(20), **Phone** VARCHAR(15), FOREIGN KEY (UID) REFERENCES User(UID)	Store information of guest that they want to register room reservation. This information includes the order number, name, birth, address, email, country, phone and account they are using for reservation.
Booking	**BookingID** INT PRIMARY KEY auto_increment, **CustomerID** INT NOT NULL, **RoomNumber** INT NOT NULL, **CheckinDate** DATE, **CheckoutDate** DATE, **NoOfDays** INT, **Status** VARCHAR(10), FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID), foreign key (RoomNumber) references	Record reservation details, including room number, check-in, check-out, and need receptionist review the booking status.

	Room(RoomNumber)	
Payment	**PaymentID** INT PRIMARY KEY auto_increment, **BookingID** INT NOT NULL, **Amoun**t DECIMAL(10, 2), **PaymentDate** DATE, **PaymentMethod** VARCHAR(50), FOREIGN KEY (BookingID) REFERENCES Booking(BookingID)	Handle payment information associated with reservation, including payment method, payment date, and payment value.

3.5. Database Data Insertion

Management table

```
INSERT INTO `Management` VALUES (1,1,'hcmiu','1234');
```

Employee table

```
INSERT INTO `Employee` VALUES (1,10,'PDM','Hotel','Group 2','manager',2000.00,'2000-11-11','2020-11-11','0882837487','hcmiu@gmail.com');
INSERT INTO `Employee` VALUES (2,10,'John','','Lee','Receptionist',NULL,'2000-05-11','2025-05-01','0564392768','Lee@gmail.com');
INSERT INTO `Employee` VALUES (3,10,'Martin','','Odegaard','Chef',NULL,'2000-05-10','2025-05-01','0536159753','o8@gmail.com');
INSERT INTO `Employee` VALUES (4,10,'Kai','','Harvert','Waiter',NULL,'2000-05-13','2025-05-01','0264563289','King@gmail.com');
INSERT INTO `Employee` VALUES (5,10,'Bukayo ','','Saka','Housekeeping',NULL,'2000-05-17','2025-05-01','0456987465','spicy@gmail.com');
INSERT INTO `Employee` VALUES (6,10,'Declan','','Rice','Reservation',NULL,'2000-05-03','2025-05-01','0365426987','100m@gmail.com');
INSERT INTO `Employee` VALUES (7,10,'William','','Saliba','Bell man',NULL,'2000-05-03','2025-05-01','0365426987','Salibo@gmail.com');
```

Hotel table

```
INSERT INTO `Hotel` VALUES (10,'HCMIU','Quarter 6, Linh Trung ward, Thu Duc, HCMC','(028) 37244270','info@hcmiu.edu.vn',5);
```

RoomType table

Room Table

```
INSERT INTO `RoomType` VALUES (1,'Superior Room','King room ',300.00,4,10);
INSERT INTO `RoomType` VALUES (2,'Superior Room','excellent',300.00,4,10);
INSERT INTO `RoomType` VALUES (3,'Queen Room','excellent',200.00,3,10);
INSERT INTO `RoomType` VALUES (4,'Queen Room','excellent',200.00,3,10);
INSERT INTO `RoomType` VALUES (5,'Single Room','good',30.00,1,10);
INSERT INTO `RoomType` VALUES (6,'Single Room','good',30.00,1,10);
INSERT INTO `RoomType` VALUES (7,'Single Room','good',30.00,1,10);
INSERT INTO `RoomType` VALUES (8,'Studio Room','good',60.00,3,10);
INSERT INTO `RoomType` VALUES (9,'Studio Room','nice',60.00,3,10);
INSERT INTO `RoomType` VALUES (10,'Deluxe Room','very nice',100.00,3,10);
INSERT INTO `RoomType` VALUES (11,'Queen Room','excellent',200.00,2,10);
INSERT INTO `RoomType` VALUES (12,'Deluxe Room','very nice',100.00,3,10);
INSERT INTO `RoomType` VALUES (13,'Deluxe Room','very nice',100.00,4,10);
INSERT INTO `RoomType` VALUES (14,'Single Room','normal',30.00,1,10);
INSERT INTO `RoomType` VALUES (15,'Single Room','normal',30.00,1,10);
INSERT INTO `RoomType` VALUES (16,'Studio Room','',60.00,2,10);
INSERT INTO `RoomType` VALUES (17,'Studio Room','good',60.00,2,10);
INSERT INTO `RoomType` VALUES (18,'Superior Room','King room',300.00,1,10);
INSERT INTO `RoomType` VALUES (19,'Deluxe Room','nice',100.00,2,10);
INSERT INTO `RoomType` VALUES (20,'Single Room','nice',30.00,2,10);
INSERT INTO `RoomType` VALUES (21,'Single Room','nice',30.00,2,10);
INSERT INTO `RoomType` VALUES (22,'Single Room','nice',30.00,2,10);
INSERT INTO `RoomType` VALUES (23,'Superior Room','good',300.00,1,10);
INSERT INTO `RoomType` VALUES (24,'Superior Room','good',300.00,1,10);
INSERT INTO `RoomType` VALUES (25,'Single Room','',30.00,1,10);
```

```
INSERT INTO `Room` VALUES (2,2,'Ready');
INSERT INTO `Room` VALUES (3,3,'Ready');
INSERT INTO `Room` VALUES (4,4,'Ready');
INSERT INTO `Room` VALUES (5,5,'Ready');
INSERT INTO `Room` VALUES (6,6,'Ready');
INSERT INTO `Room` VALUES (7,7,'Ready');
INSERT INTO `Room` VALUES (8,8,'Ready');
INSERT INTO `Room` VALUES (9,9,'Ready');
INSERT INTO `Room` VALUES (10,10,'Ready');
INSERT INTO `Room` VALUES (11,11,'Ready');
INSERT INTO `Room` VALUES (12,12,'Ready');
INSERT INTO `Room` VALUES (13,13,'Ready');
INSERT INTO `Room` VALUES (14,14,'Ready');
INSERT INTO `Room` VALUES (15,15,'Used');
INSERT INTO `Room` VALUES (16,16,'Ready');
INSERT INTO `Room` VALUES (17,17,'Ready');
INSERT INTO `Room` VALUES (18,18,'Ready');
INSERT INTO `Room` VALUES (19,19,'Ready');
INSERT INTO `Room` VALUES (20,20,'Ready');
INSERT INTO `Room` VALUES (21,21,'Cleaning');
INSERT INTO `Room` VALUES (22,22,'Ready');
INSERT INTO `Room` VALUES (23,23,'Ready');
INSERT INTO `Room` VALUES (24,24,'Cleaning');
INSERT INTO `Room` VALUES (25,25,'Cleaning');
```

Customer Table

```
INSERT INTO `Customer` VALUES (4,4,'Gabriel Jesus','2000-05-16','Arsenal','js@gmail.com','Brazil','0564324643');
INSERT INTO `Customer` VALUES (5,4,'Thomas Partey','2000-05-21','Arsenal','TP@gmail.com','Ghana','0546982317');
INSERT INTO `Customer` VALUES (6,5,'Trinh Tran Phuong Tuan','1997-04-12','152 Tran Phu, Ben Tre','J97@gmail.com','Vietnam','09873628918');
INSERT INTO `Customer` VALUES (7,4,'Gabriel Martinelli','2000-05-17','Arsenal','GM@gmail.com','Brazil','0569846231');
INSERT INTO `Customer` VALUES (19,1,'Viet Nam','2000-05-11','Thu Duc','abc@gmail.com','Vietnam','0192836475');
INSERT INTO `Customer` VALUES (23,4,'Zichenko','2000-05-15','Arsenal','z30@gmail.com','Afghanistan','0564978356');
INSERT INTO `Customer` VALUES (25,2,'William','1980-01-01','LA','william@gmail.com','United States','0123524168');
INSERT INTO `Customer` VALUES (26,4,'Gabriel','2000-05-01','Arsenal','G@gmail.com','Brazil','569846231');
INSERT INTO `Customer` VALUES (27,4,'Jorginho','2000-05-22','Arsenal','J20@gmail.com','Italy','0579862341');
INSERT INTO `Customer` VALUES (28,4,'Benjamin White','2000-05-16','Arsenal','b4@gmail.com','United Kingdom','0453261789');
INSERT INTO `Customer` VALUES (29,4,'David Raya','2000-05-10','Arsenal','gk@gmail.com','Spain','0154693741');
INSERT INTO `Customer` VALUES (30,2,'Alice','1989-10-25','Pakis','a@gmail.com','Pakistan','3216410629');
INSERT INTO `Customer` VALUES (31,5,'Eric Krylova','1985-12-26','Vico Giganti, 13, 06049-Bazzano Inferiore PG','EricKrylova@jourrapide.com','Russian Federation','+39 366 0158925');
INSERT INTO `Customer` VALUES (32,5,'Nguyen Hoang Bao','2005-05-26','15A Nguyen Cu Trinh, Q1, HCMC','NguyenHoangBao@gmail.com','Vietnam','0908497879');
INSERT INTO `Customer` VALUES (33,5,'Rahmana Viderman','1998-01-09','3535 Driftwood Road Oakland, CA 94612','RahmanaViderman@jourrapide.com','United States','+1 408-403-7158');
INSERT INTO `Customer` VALUES (34,5,'Harald Andreuzzi','2004-11-29','2311 Jerry Toth Drive Kodiak, AK 99615','HaraldAndreuzzi@dayrep.com','United States','+1 907-486-9747');
INSERT INTO `Customer` VALUES (35,5,'Julia Kaitainen','1987-11-02','51, place de Miremont 47300 VILLENEUVE-SUR-LOT','JuliaKaitainen@armyspy.com','France','+33576561911');
INSERT INTO `Customer` VALUES (36,5,'Mee Pan','1992-01-27','73, rue du Gue Jacquet 36000 CHÂTEAUROUX','MeePan@teleworm.us','France','+33218412541');
INSERT INTO `Customer` VALUES (37,8,'Rinkimirikakuta','1997-02-06','Belgrád rkp. 25.','Rinkimirikakuta@teleworm.us','Hungary','9434795');
INSERT INTO `Customer` VALUES (38,8,'Bowman Brownlock','1950-03-15','3876 rue Parc Sherbrooke, QC J1H 1R6','BowmanBrownlock@dayrep.com','Canada','8195789160');
INSERT INTO `Customer` VALUES (39,9,'Johny Deep','1999-08-01','789 St.Arson ','JD99@gmail.com','USA','+58783572');
INSERT INTO `Customer` VALUES (40,9,'Duy','2003-02-09','237 nguyen xi','jj7777@gmail.com','Vietnam','0837213458');
```

Booking Table

```sql
INSERT INTO `Booking` VALUES (1,4,2,'2025-05-09','2025-05-17',8,'Confirm');
INSERT INTO `Booking` VALUES (2,5,10,'2025-05-08','2025-05-15',7,'Confirm');
INSERT INTO `Booking` VALUES (3,6,5,'2025-05-21','2025-05-25',4,'Confirm');
INSERT INTO `Booking` VALUES (4,7,10,'2025-05-09','2025-05-16',7,'Confirm');
INSERT INTO `Booking` VALUES (7,19,2,'2025-05-18','2025-05-20',2,'Confirm');
INSERT INTO `Booking` VALUES (8,23,5,'2025-05-09','2025-05-15',6,'Confirm');
INSERT INTO `Booking` VALUES (9,25,13,'2025-05-27','2025-05-31',4,'Confirm');
INSERT INTO `Booking` VALUES (10,26,5,'2025-05-02','2025-05-08',6,'Confirm');
INSERT INTO `Booking` VALUES (11,27,5,'2025-05-16','2025-05-23',7,'Confirm');
INSERT INTO `Booking` VALUES (12,28,10,'2025-05-09','2025-05-21',12,'Confirm');
INSERT INTO `Booking` VALUES (13,29,20,'2025-05-01','2025-05-13',12,'Confirm');
INSERT INTO `Booking` VALUES (14,30,2,'2025-05-23','2025-05-26',3,'Confirm');
INSERT INTO `Booking` VALUES (15,31,3,'2025-05-19','2025-05-25',6,'Confirm');
INSERT INTO `Booking` VALUES (16,32,5,'2025-07-14','2025-07-20',6,'Confirm');
INSERT INTO `Booking` VALUES (17,33,5,'2025-05-19','2025-05-25',6,'Confirm');
INSERT INTO `Booking` VALUES (18,34,8,'2025-08-18','2025-08-21',3,'Confirm');
INSERT INTO `Booking` VALUES (19,35,18,'2025-05-19','2025-05-21',2,'Confirm');
INSERT INTO `Booking` VALUES (20,36,11,'2025-09-11','2025-09-20',9,'Confirm');
INSERT INTO `Booking` VALUES (21,37,8,'2025-05-20','2025-05-22',2,'Pending');
INSERT INTO `Booking` VALUES (22,38,18,'2025-05-21','2025-05-25',4,'Confirm');
INSERT INTO `Booking` VALUES (23,39,18,'2025-05-18','2025-05-22',4,'Confirm');
INSERT INTO `Booking` VALUES (24,40,10,'2025-05-22','2025-05-31',9,'Confirm');
INSERT INTO `Booking` VALUES (26,42,19,'2025-05-20','2025-05-26',6,'Confirm');
INSERT INTO `Booking` VALUES (27,42,19,'2025-05-20','2025-05-26',6,'Confirm');
INSERT INTO `Booking` VALUES (28,44,20,'2025-05-20','2025-05-23',3,'Confirm');
```

Payment Table

```
INSERT INTO `Payment` VALUES (5,3,320.00,'2025-05-18','Card');
INSERT INTO `Payment` VALUES (6,4,1750.00,'2025-05-17','Cash');
INSERT INTO `Payment` VALUES (7,1,4000.00,'2025-05-18','Card');
INSERT INTO `Payment` VALUES (8,2,1750.00,'2025-05-18','Card');
INSERT INTO `Payment` VALUES (9,7,1000.00,'2025-05-18','Card');
INSERT INTO `Payment` VALUES (11,8,480.00,'2025-05-16','Card');
INSERT INTO `Payment` VALUES (12,9,1200.00,'2025-05-23','Cash');
INSERT INTO `Payment` VALUES (13,10,480.00,'2025-05-23','Cash');
INSERT INTO `Payment` VALUES (14,11,560.00,'2025-05-22','Cash');
INSERT INTO `Payment` VALUES (15,12,3000.00,'2025-05-22','Cash');
INSERT INTO `Payment` VALUES (16,13,1560.00,'2025-05-22','Cash');
INSERT INTO `Payment` VALUES (17,14,1500.00,'2025-05-22','Membership');
INSERT INTO `Payment` VALUES (18,15,2100.00,'2025-05-21','Card');
INSERT INTO `Payment` VALUES (19,16,480.00,'2025-05-29','Cash');
INSERT INTO `Payment` VALUES (20,17,480.00,'2025-05-23','Cash');
INSERT INTO `Payment` VALUES (21,18,630.00,'2025-05-25','Cash');
INSERT INTO `Payment` VALUES (22,19,700.00,'2025-05-25','Membership');
INSERT INTO `Payment` VALUES (23,20,2700.00,'2025-05-25','Card');
INSERT INTO `Payment` VALUES (24,23,1400.00,'2025-05-18','Card');
INSERT INTO `Payment` VALUES (25,24,2250.00,'2025-05-20','Cash');
INSERT INTO `Payment` VALUES (26,28,390.00,'2025-05-19','Membership');
INSERT INTO `Payment` VALUES (27,26,1200.00,'2025-05-25','Cash');
INSERT INTO `Payment` VALUES (28,27,1200.00,'2025-05-31','Cash');
INSERT INTO `Payment` VALUES (29,27,1200.00,'2025-05-21','Card');
INSERT INTO `Payment` VALUES (31,22,1400.00,'2025-05-25','Card');
INSERT INTO `Payment` VALUES (32,22,1400.00,'2025-05-25','Card');
```

User Table

```
INSERT INTO `User` VALUES (1,'Manager','nothing','!@#$%^&*()Qwerty123');
INSERT INTO `User` VALUES (2,'nam','test@gmail.com','123');
INSERT INTO `User` VALUES (3,'vietnam','yuenan@gmail.com','abcxyz');
INSERT INTO `User` VALUES (4,'Duy','asasa@gmail.com','123456');
INSERT INTO `User` VALUES (5,'PhuongTran','nypd.vn.swat@gmail.com','1234');
INSERT INTO `User` VALUES (6,'Tommy','12345@gmail.com','12345');
INSERT INTO `User` VALUES (7,'abcxyz','abcxyz@gmail.com','abcxyz');
INSERT INTO `User` VALUES (8,'man','tset-3002@gmail.com','123');
INSERT INTO `User` VALUES (9,'Johny','nganhminh23@gmail.com','01qD0110');
INSERT INTO `User` VALUES (10,'Nghia','nghia123@gmail.com','nguyenchinhnghia');
INSERT INTO `User` VALUES (11,'Tien','T789@gmail.com','Tien');
INSERT INTO `User` VALUES (12,'Khang12345','khang1@gmail.com','12345');
```

3.6. Database Query

3.6.1. Query Analysis

Most queries shown below are in the same form of this structure to ensure the user input is sanitized, where each question mark shows it is parameterized to suit the content user data:

- **INSERT INTO VALUES:** to insert data into the tables
- **SELECT:** to show the columns for return
- **FROM:** to specify the table use
- **INNER JOIN:** to join the table for more linked data query
- **WHERE:** to set a condition for each query
- **ORDER BY:** to sort the results in ascending order
- **GROUP BY:** to pack the same data into groups

3.6.2. Web Interface

a.Login Page:

Check login authentication.

```
SELECT * FROM User WHERE Email = '$Email' AND Password ='$Password'
```

Login successfully, create a new session value by email.

```
$_SESSION['usermail'] = $Email
```

b.Register account:

Check username or email exist.

```
SELECT * FROM User WHERE Email = '$Email' OR Username = '$Username'
```

Register the new one if username or email available.

```
INSERT INTO User (Username, Email, Password) VALUES ('$Username', '$Email', '$Password')
```

c.Reservation:

After login successfully, website get the User ID of this account.

```
SELECT UID FROM User WHERE email= '".$_SESSION['usermail']."'
```

$_SESSION["UID"] is assigned to $GetUID

```
$uid = $row["UID"];
$_SESSION["UID"] = $uid;
```

Retrieve the data if the room type is available.

Note: **(%d)**= Integer value

```
SELECT Room.RoomNumber FROM Room, RoomType WHERE Room.Status='Ready' AND
RoomType.Name='$RoomType' AND Room.TypeID=RoomType.TypeID AND
RoomType.Capacity=(%d)
```

The type of room is available, record the new customer information for this.

```
INSERT INTO Customer (UID, CName, DateOfBirth, Address, Email, Country, Phone) VALUES
($getUID, '$Name', '$Birth', '$Address', '$Email', '$Country', '$Phone')
```

Retrieve CustomerID after inserting the new record.

```
SELECT CustomerID FROM Customer WHERE Email = '$Email' AND Phone = '$Phone'
```

Link CustomerID to Booking table and make new reservation.

```
INSERT INTO Booking
(CustomerID,RoomNumber,CheckinDate,CheckoutDate,NoOfDays,Status) VALUES
($nocid,'$RoomAvailable','$cin','$cout',datediff('$cout','$cin')
```

3.6.3. Dashboard

Retrieve data what type of room were booked by guest.

```
SELECT Booking.BookingID, Customer.CName, Customer.Email,
Customer.Country, Customer.Phone, Customer.Address, RoomType.Name,
RoomType.Capacity, Booking.CheckinDate, Booking.CheckoutDate,
Booking.NoOfDays, Booking.Status FROM Booking, Customer, Room,
RoomType WHERE Booking.CustomerID = Customer.CustomerID AND
Customer.UID = $s_uid AND Booking.RoomNumber = Room.RoomNumber
AND Room.TypeID = RoomType.TypeID
```

If the reservation has not been approved(status: pending), user can cancel the reservation. It is executed the query respectively.

+Get BookingID from the chosen booking.

```
<a href='roombookdelete.php?id=". $res['BookingID'] ."'><button class='btn
btn-danger'>Cancel</button>
```

+Pass BookingID value and get CustomerID what reservation is pending.

```
SELECT CustomerID FROM Booking WHERE Booking.BookingID = $bookid
AND Booking.Status = 'Pending'
```

+Assign CID value by CustomerID

```
$cid = (int)$row["CustomerID"];
```

+Delete record from Booking table.

```
DELETE FROM Booking WHERE Booking.BookingID = $bookid
```

+Delete record from Customer table

```
DELETE FROM Customer WHERE Customer.CustomerID = $cid
```

3.6.4. Application Interface

3.6.4.1. Login interface:

Check manager credential.

```
SELECT * FROM Management WHERE EmpUsername='?' AND
PASSWORD='?'
```

3.6.4.2. Dashboard management:

Show employee name who is logged in.

```
SELECT Employee.LastName FROM HMS.Employee WHERE
Employee.EmployeeID IN (SELECT EmployeeID FROM Management
WHERE EmpUsername = '?')
```

- **Manage employee:**

Retrieve employee data in the table.

```
SELECT EmployeeID, FirstName, MiddleName, LastName, Role, DateOfBirth,
HireDate, Phone, Email FROM HMS.Employee
```

Add a new employee.

```
insert into HMS.Employee (FirstName, MiddleName, LastName, Role,
DateOfBirth, HireDate, Phone, Email) values ('?','?','?','?','?','?','?','?')
```

Remove a employee.

```
DELETE FROM HMS.Employee WHERE EmployeeID=? AND
Role!='Manager'
```

Modify a employee information.

```
UPDATE HMS.Employee SET
        FirstName='?',
        MiddleName='?',
        LastName='?',
        Role='?',
        DateOfBirth='?'
        HireDate='?',
        Phone='?',
        Email='?'
      WHERE EmployeeID=?
```

3.6.4.3. Manage RoomType:

Retrieve data into the table.

```
SELECT RoomType.TypeID, RoomType.Name, RoomType.Description,
RoomType.Price, RoomType.Capacity, Room.RoomNumber, Room.Status
FROM RoomType, Room WHERE RoomType.TypeID = Room.TypeID
```

Add a new available room for the hotel.

```
insert into HMS.RoomType (Name, Description, Price, Capacity) value
('?','?','?','?')
```

```
insert into Room(TypeID) value ((select max(RoomType.TypeID) from
RoomType ORDER BY RoomType.TypeID DESC LIMIT 1))
```

Remove a room for hotel.

```
DELETE FROM HMS.Room WHERE RoomNumber=? AND Status!='Used'
```

Update room status.

```
update Room set Status ='?' where RoomNumber = ?
```

3.6.4.4. **Booking**:

Add a new reservation:

+Insert a record about a new customer.

```
insert into HMS.Customer (UID, CName, DateOfBirth, Address, Email,
Country, Phone) values (1,'?','?','?','?','?','?'")
```

Note: **UID = 1** means manager create a new customer information due to UID cannot be blank.

+Get CustomerID after adding a current customer information.

```
Select CustomerID from Customer WHERE CName = '?' AND DateOfBirth =
'?' AND Address = '?' AND Email = '?' AND Country = '?' AND Phone='?
```

+Insert a record about reservation for new customer after getting CustomerID.

```
insert into HMS.Booking (CustomerID, RoomNumber, CheckinDate,
CheckoutDate, NoOfDays, Status) values (?,?,'?','?',?,'? ')
```

Edit a reservation:

+Get data from Booking, Customer when clicked into the exact row of the table.

```
SELECT Customer.CName, Customer.DateOfBirth, Customer.Address,
Customer.Email, Customer.Country, Customer.Phone, Booking.RoomNumber,
Booking.NoOfDays, Booking.CheckinDate, Booking.CheckoutDate,
Booking.Status FROM Booking, Customer, Room, RoomType

WHERE Booking.CustomerID = Customer.CustomerID

AND Booking.RoomNumber = Room.RoomNumber

AND Room.TypeID = RoomType.TypeID
```

```
AND Booking.BookingID = model.getValueAt(getRow(), 0))
```

Get BookingID for

```
select BookingID from HMS.Booking where CustomerID = ?
```

Remove booking

```
delete from HMS.Booking where Booking.BookingID =?
```

<u>Addition:</u> If the reservation status is set to be confirmed, the exact value (BookingID) will be inserted into the Payment table as a new record.

```
select COUNT(*) AS RowRecord from Payment where BookingID =
model.getValueAt(getRow(), 0)
```

In case of reservation status is set to be another mode (except **Confirm**), the Payment table will automatically delete the record which has the exact BookingID where the employee clicked in the table.

```
delete from Payment where BookingID = model.getValueAt(getRow(), 0)
```

3.6.4.5. Payment:

Choosing the record in the Payment table:

```
select RoomType.Price, RoomType.Capacity, RoomType.Description,
Booking.NoOfDays from RoomType, Room, Booking, Payment where
Booking.BookingID = ? and Room.RoomNumber =
Booking.RoomNumber and Room.TypeID = RoomType.TypeID
```

Update the payment method:

```
update Payment set Amount = ?, PaymentDate = ? , PaymentMethod =
'?' where Payment.PaymentID = ?
```

CHAPTER 4. CONNECTION BETWEEN SQL AND JAVA

Due to the need for synchronization, the team has set up a local host running MySQL on Windows.

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package db;
import java.sql.*;
/**
 *
 * @author WOW
 */
public class connectdb {
    public static Connection getCon(){
        try{
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/HMS?useSSL=false","root","123456");
            return con;
        }
            catch (Exception e){
                    return null;
                    }
    }
}
```

Figure 4: *Connection Database via JDBC*

This is how to connect to a SQL database using Java, using the MySQL Java module.

SQL server connection links to application and HTTP website. The group server's IP address is **localhost**, port is **3306**, **HMS** is the database name, **root** is the user credential, **123456** is the server's MySQL password.
Web interface part ⇒ the team uses PHP to connect to the database.

```php
<?php

$server = "localhost";
$username = "root";
$password = "123456";
$database = "HMS";

$conn = mysqli_connect($server,$username,$password,$database);

if(!$conn){
    die("<script>alert('connection Failed.')</script>");
} ?>
```

Figure 5: *Database Connection via PHP*

PHP has the MySQL library ⇐ easily connects to the server's database.

CHAPTER 5. DEMO PROJECT

5.1. The user experience:

5.1.1. Overview:

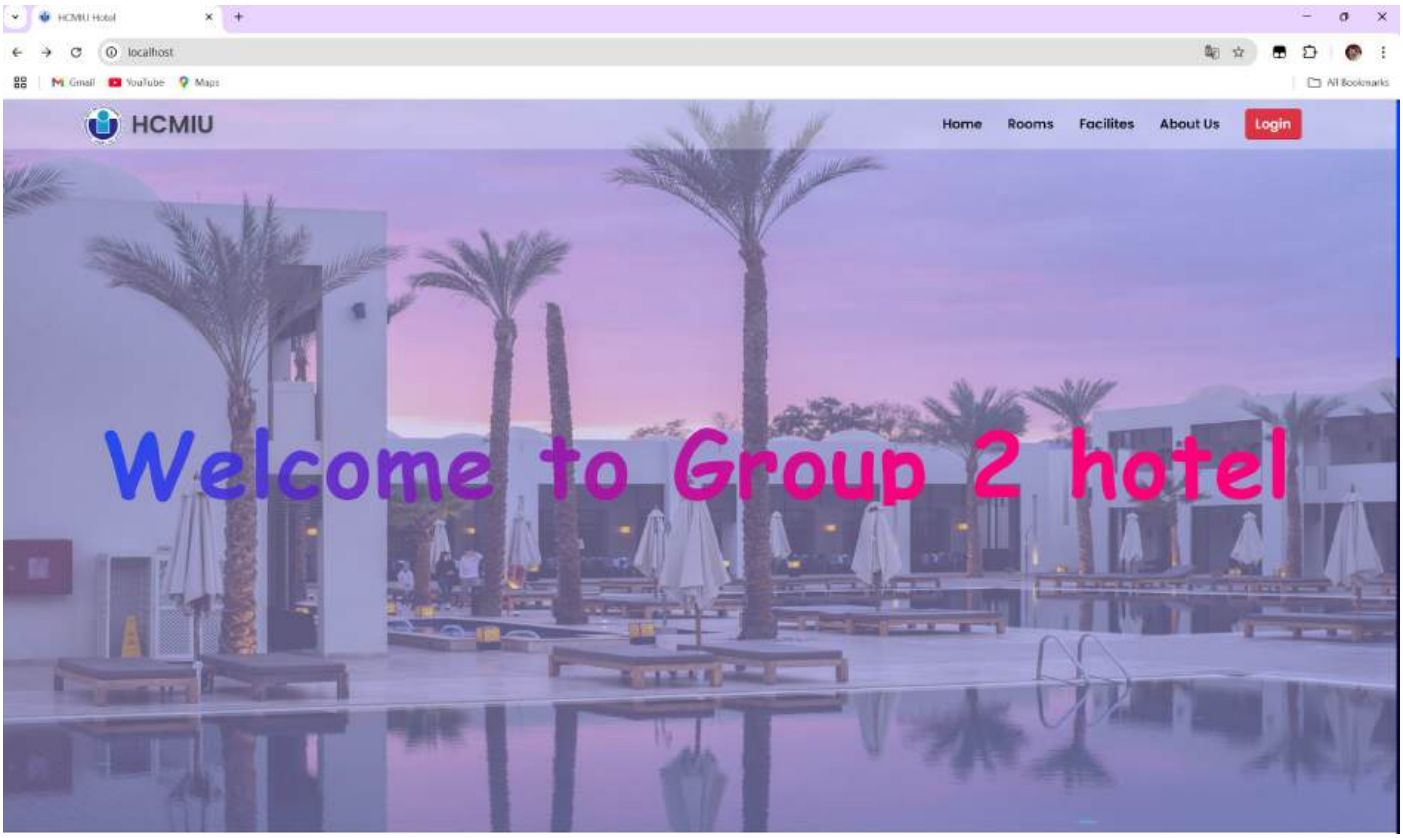


Figure 6: *The index page*

Now click on the Login button to move on the Login/Register to create a new account or login into the website.

5.1.2. Register user:

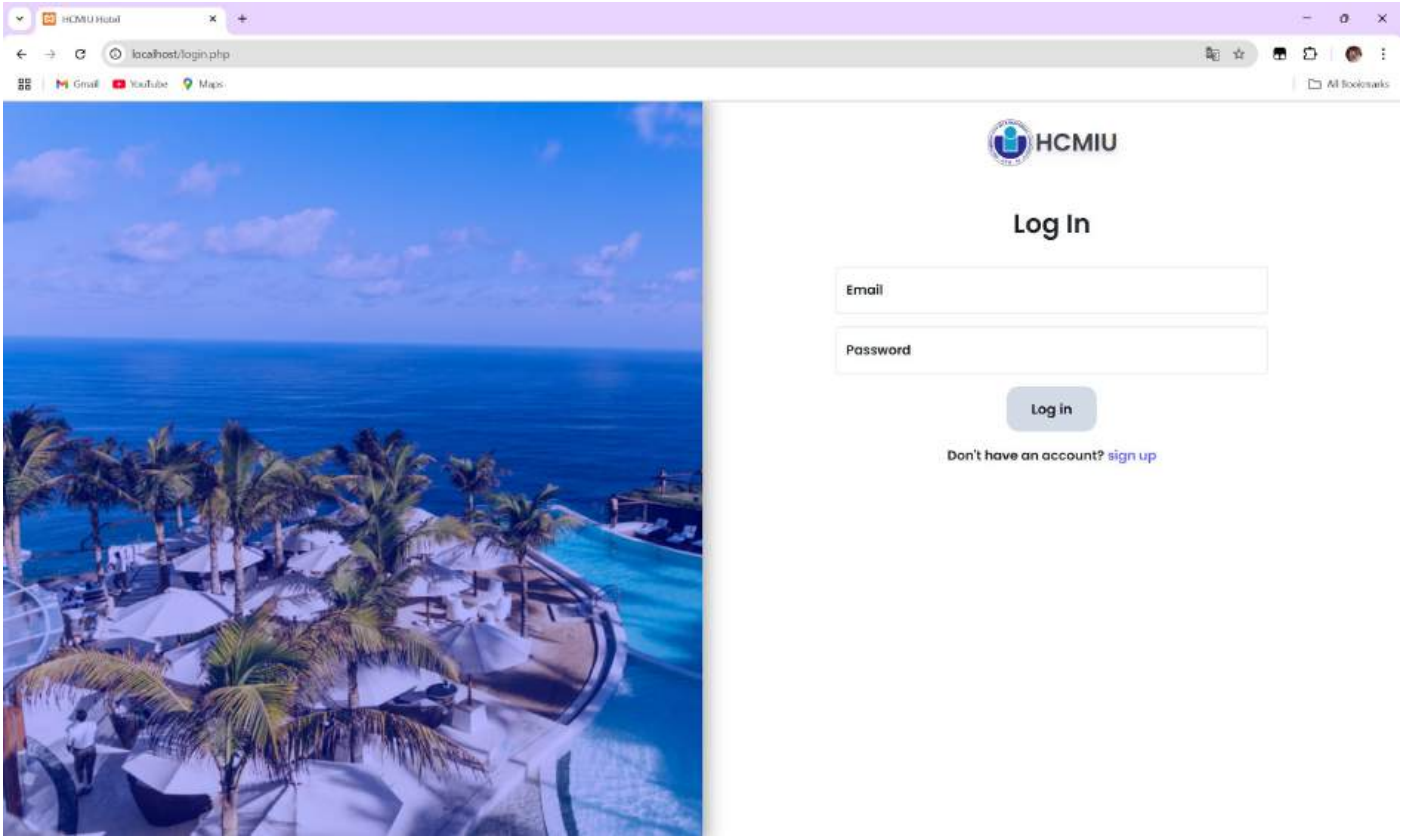


Figure 7: *Login site*

Click on the “sign up”, the registration form appeared. Fill the details into the registration form and then click the “Sign up" button.

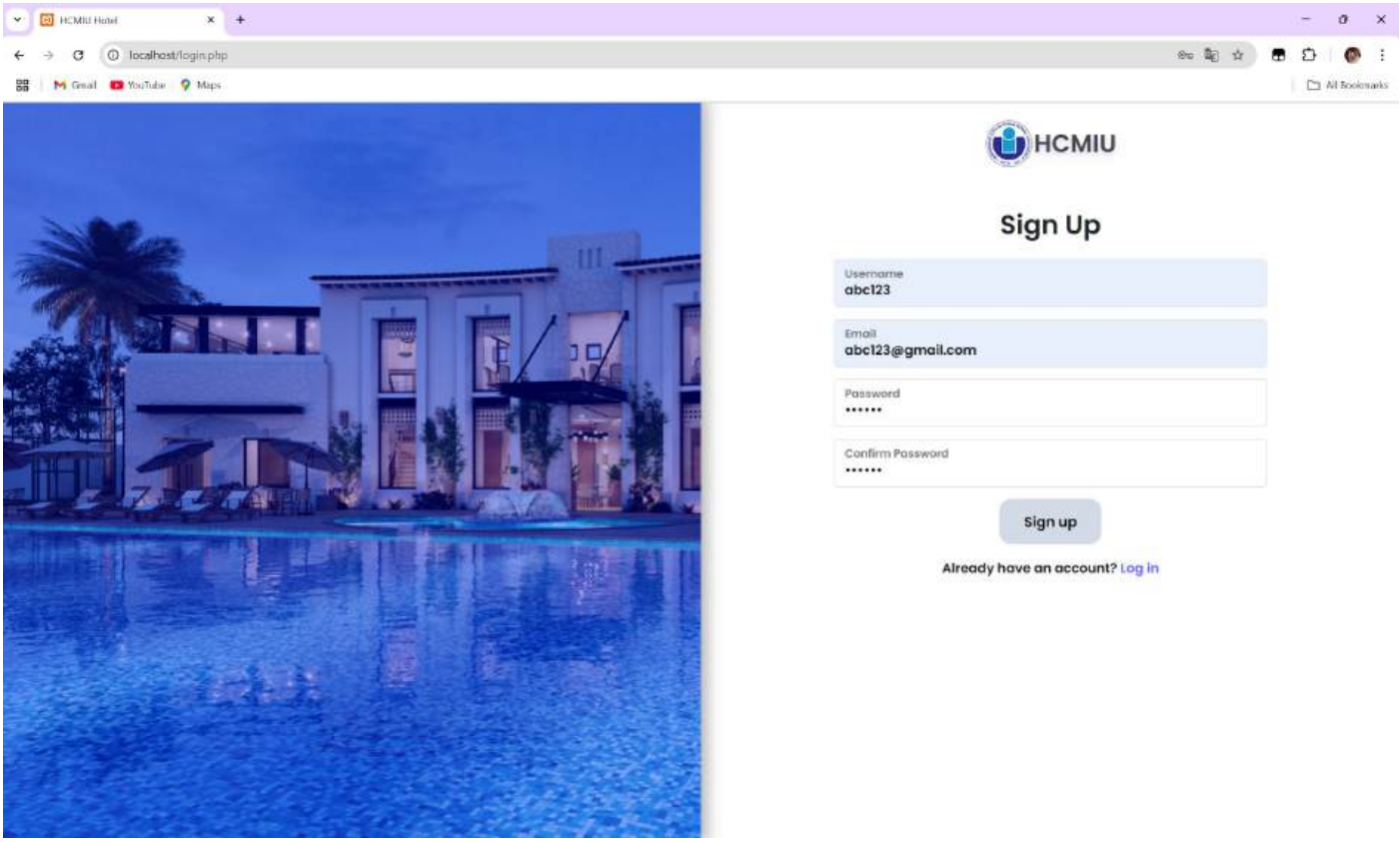


Figure 8: *Register account*

5.1.3. Booking:

After registering, the session of user is stored in the browser, now the user begins to book the reservation.

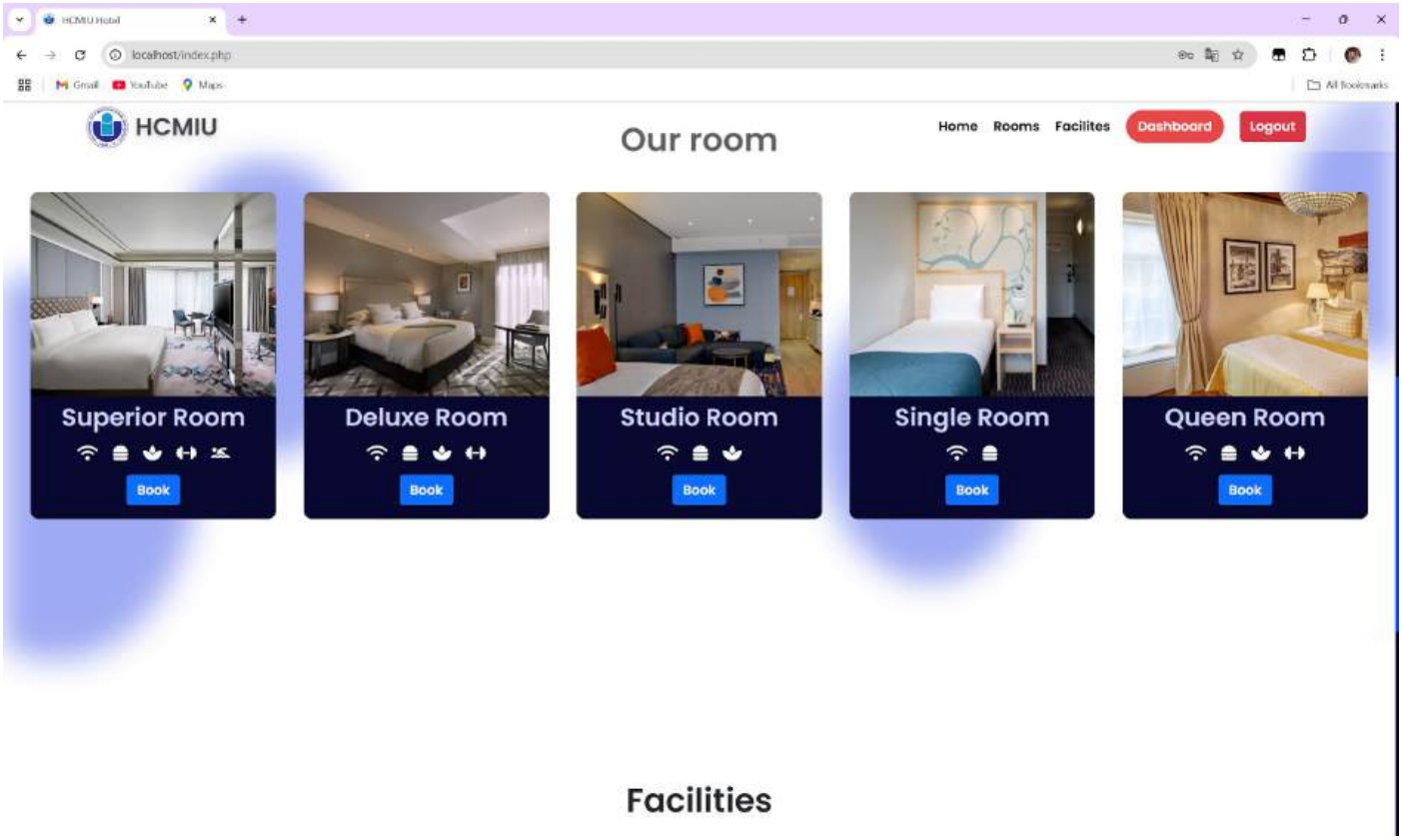


Figure 9: *Scroll down to the list of room type to book the reservation*

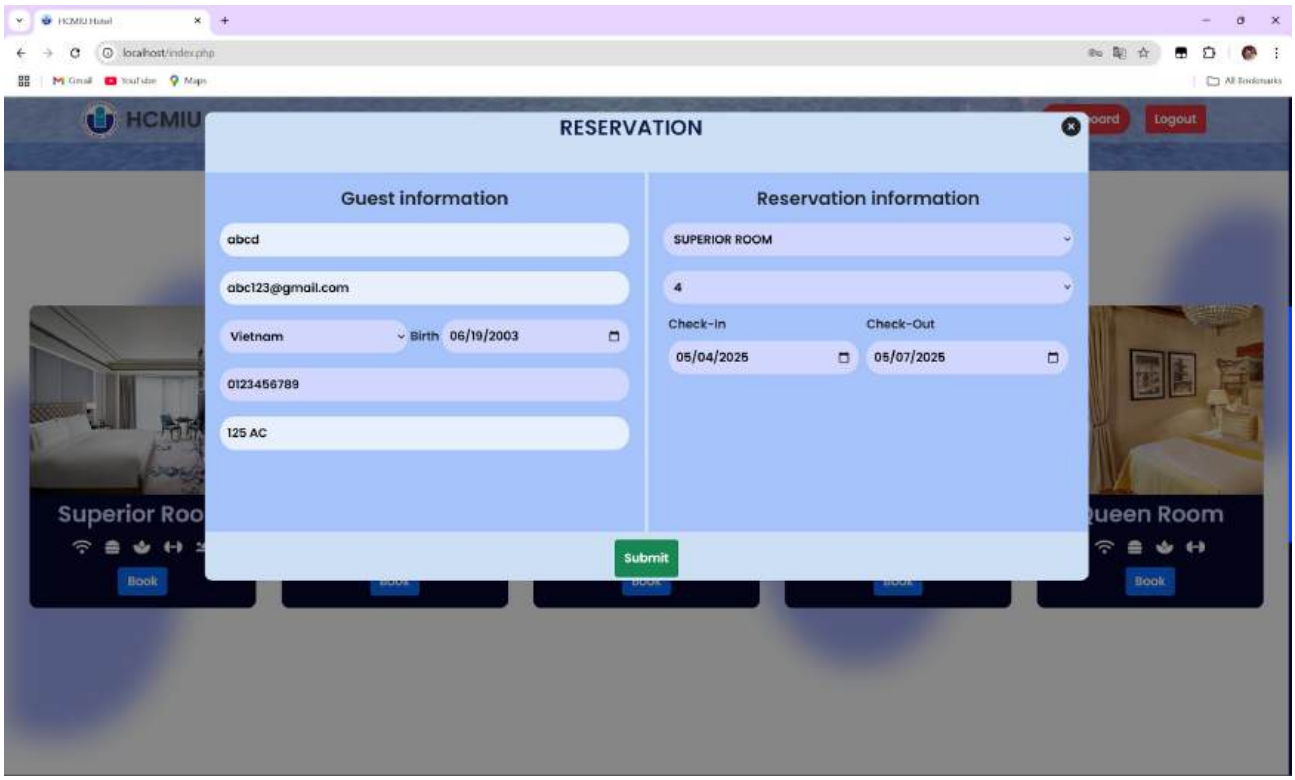


Figure 10: *Fill the information for reservation*

In case of the room type available, the notification panel will be shown like the image as Figure 11 below.

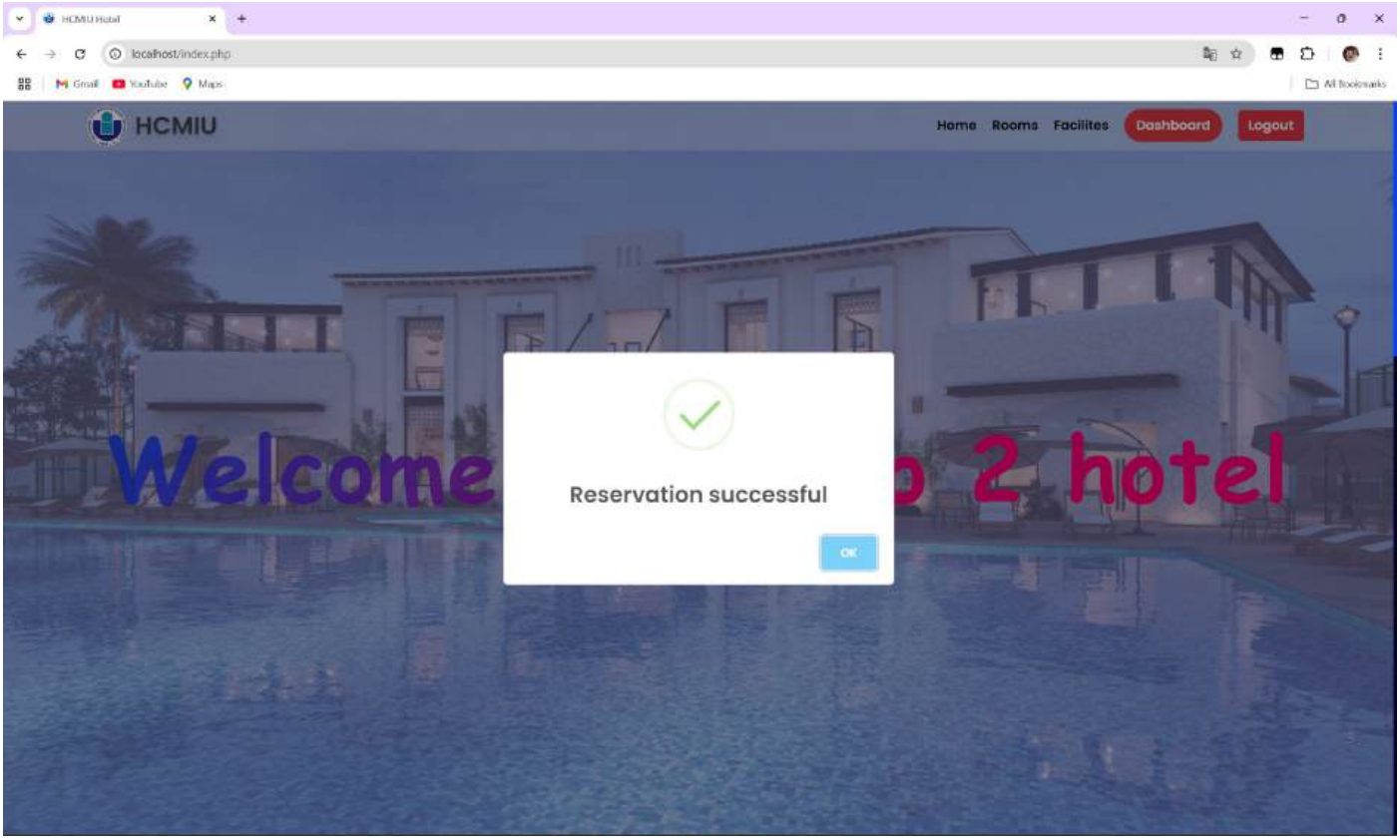


Figure* 11*: *Reservation successful*

With the room type unavailable, the notification panel will be shown like the image below.

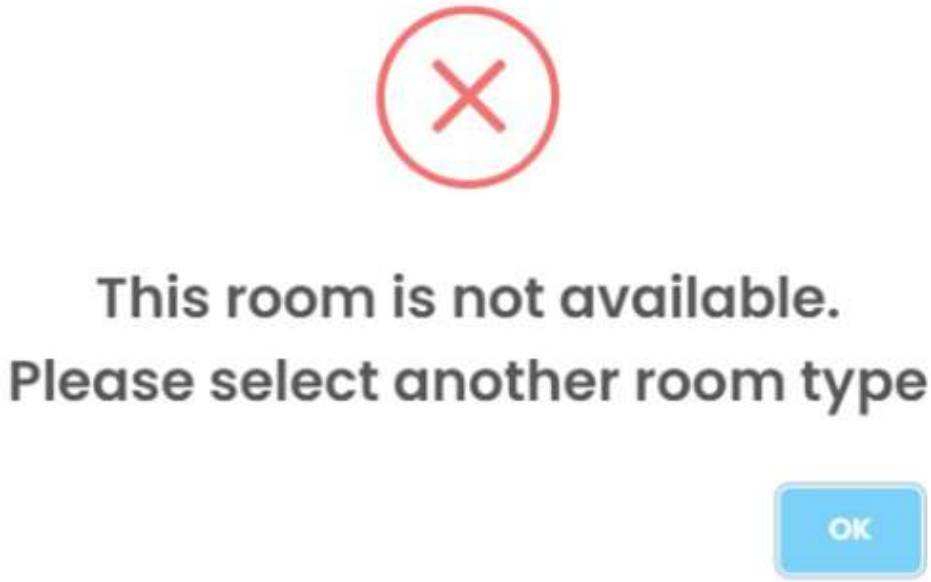


Figure 12: *Failure reservation due to room type unavailable*

5.1.4. Dashboard:

After making a reservation successfully, we return to the main index page and click on the Dashboard button to know what reservation has been booked.

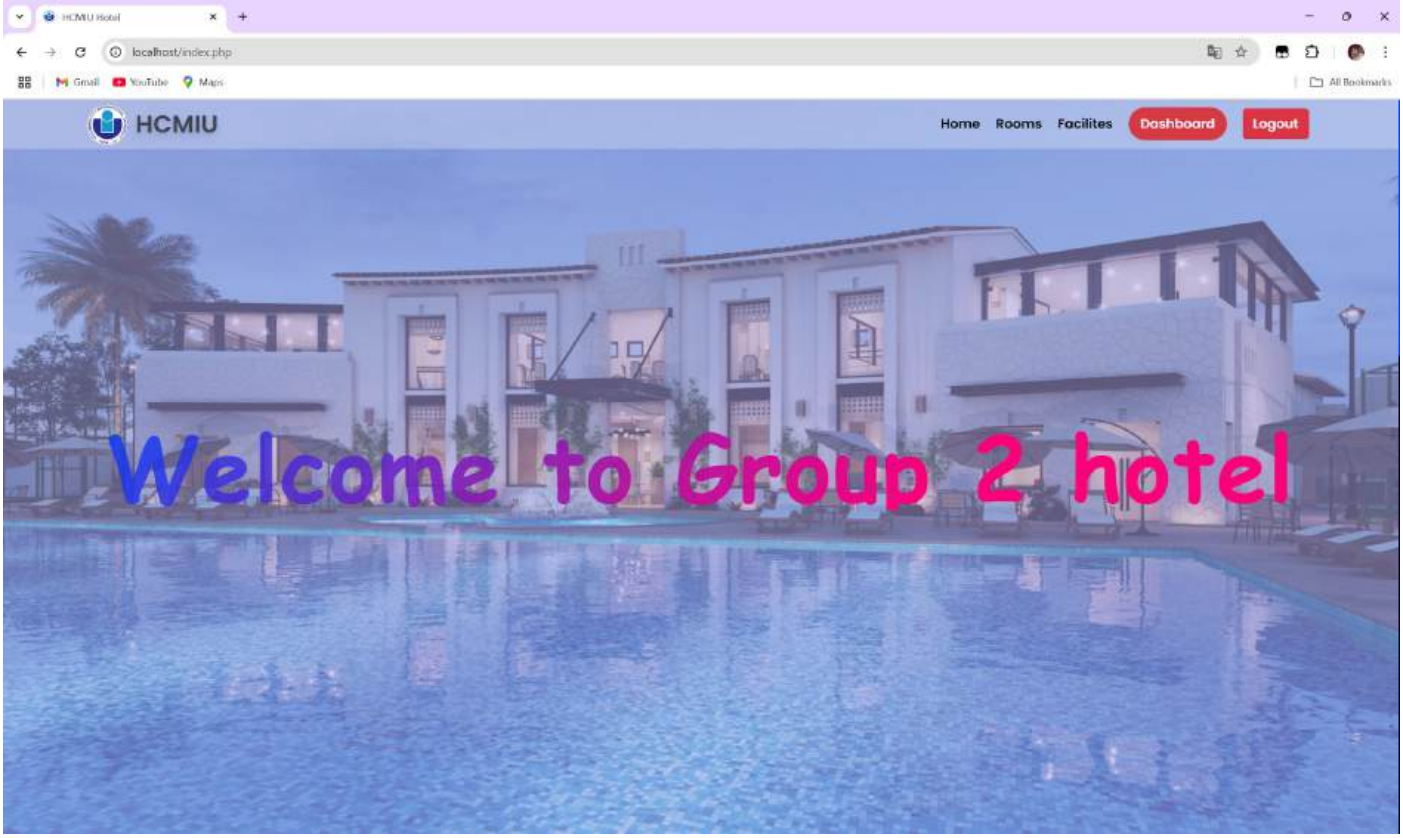


Figure 13: *The main index site*

Figure 14*: Dashboard site*

After booking, the reservation status is pending, so user can cancel the reservation if user do not want to book this reservation anymore.

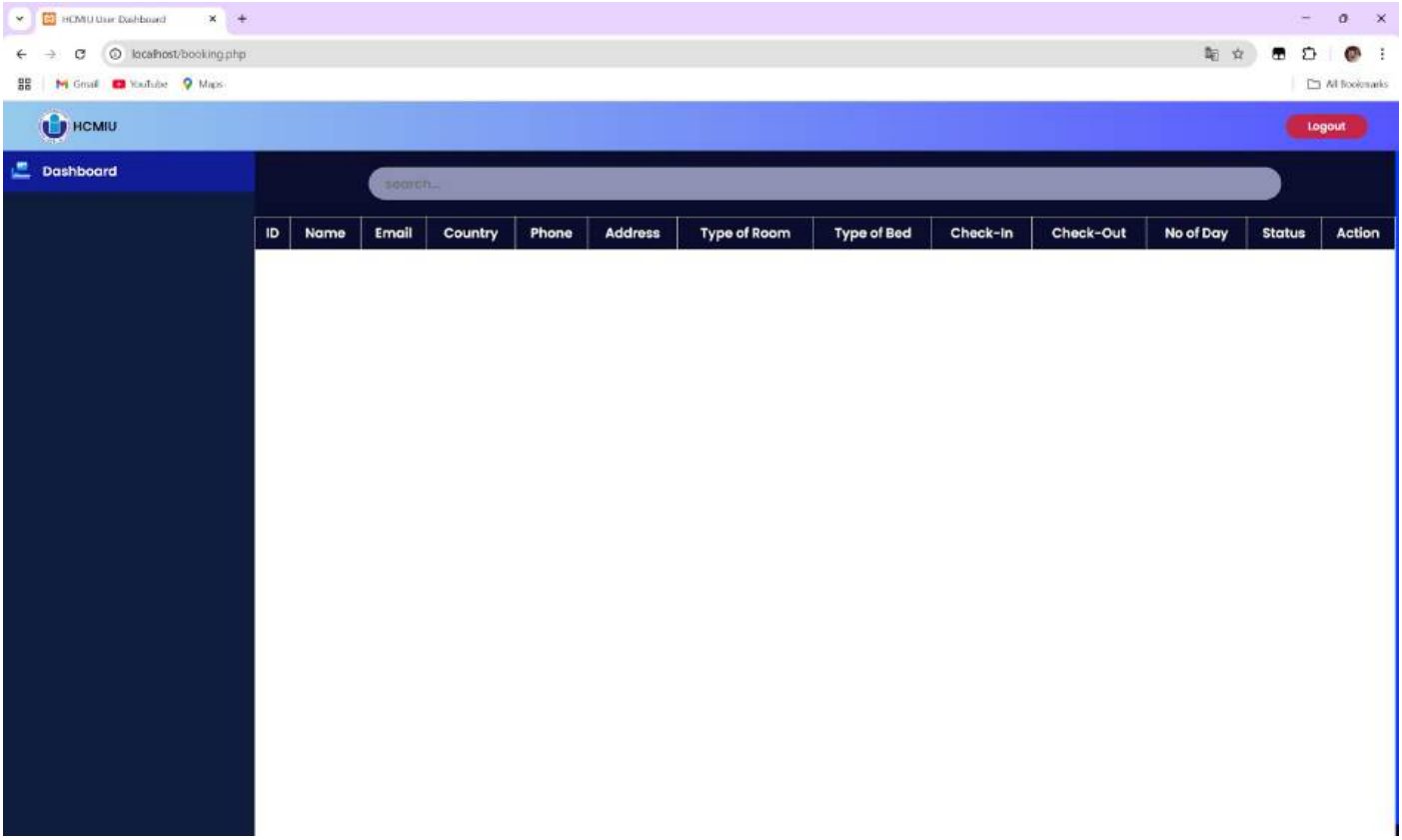


Figure 15: *Cancel reservation successfully*

If the booking is confirmed by the employee through Java application after booking on this website, the user cannot see the reservation information in the dashboard site.

5.2. The hotel management system application:

5.2.1. Login:

The manager uses the application to manage the hotel. This application has 4 functions such as: Manage employee, manage room/roomtype, booking, and payment.

This is the manager credential:

Username: hcmiu

Password: 1234

Figure 16: *Login interface*

For the question about manager credentials forgotten, contact the database developer to create a new account or recover the credentials.

5.2.2. Dashboard:

After login successfully, dashboard interface is shown as image below.

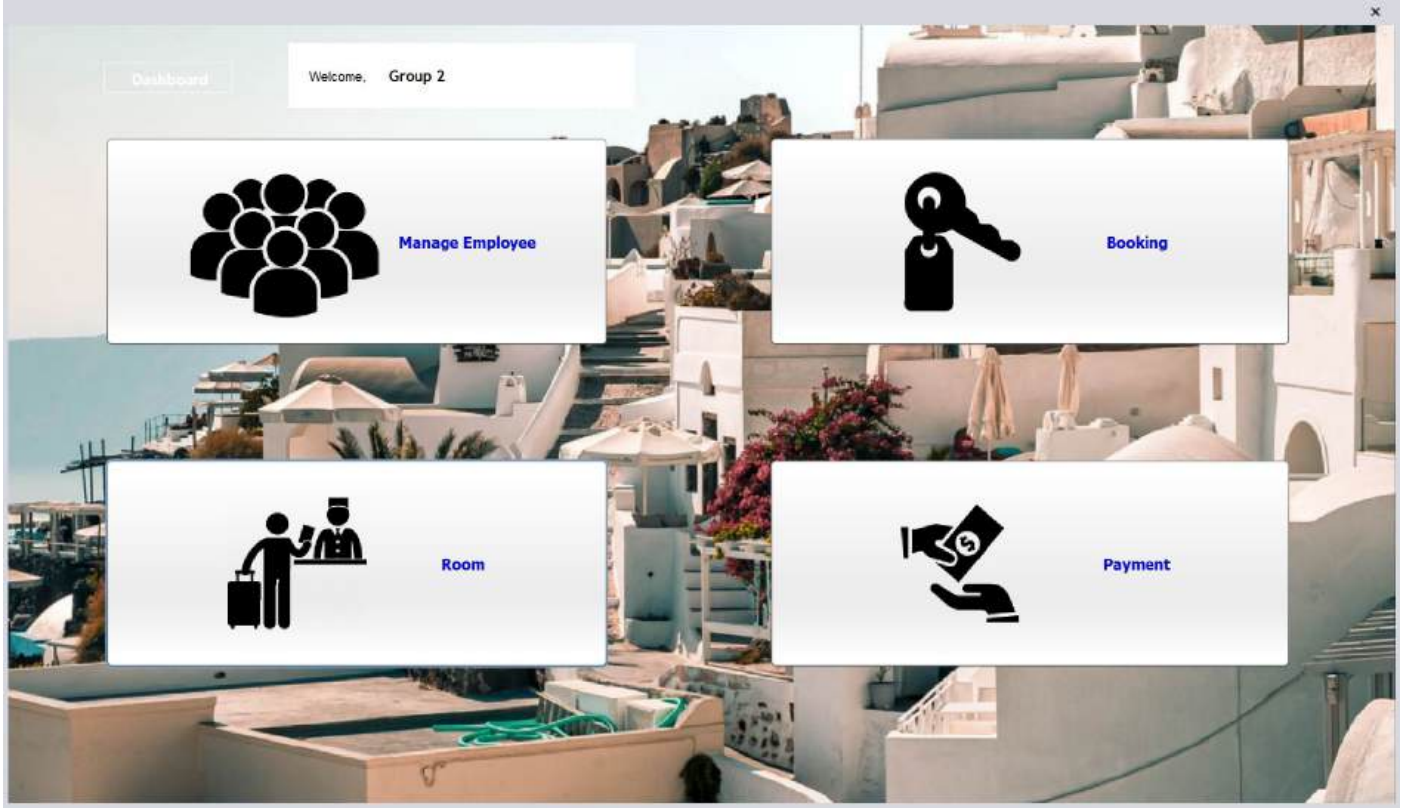
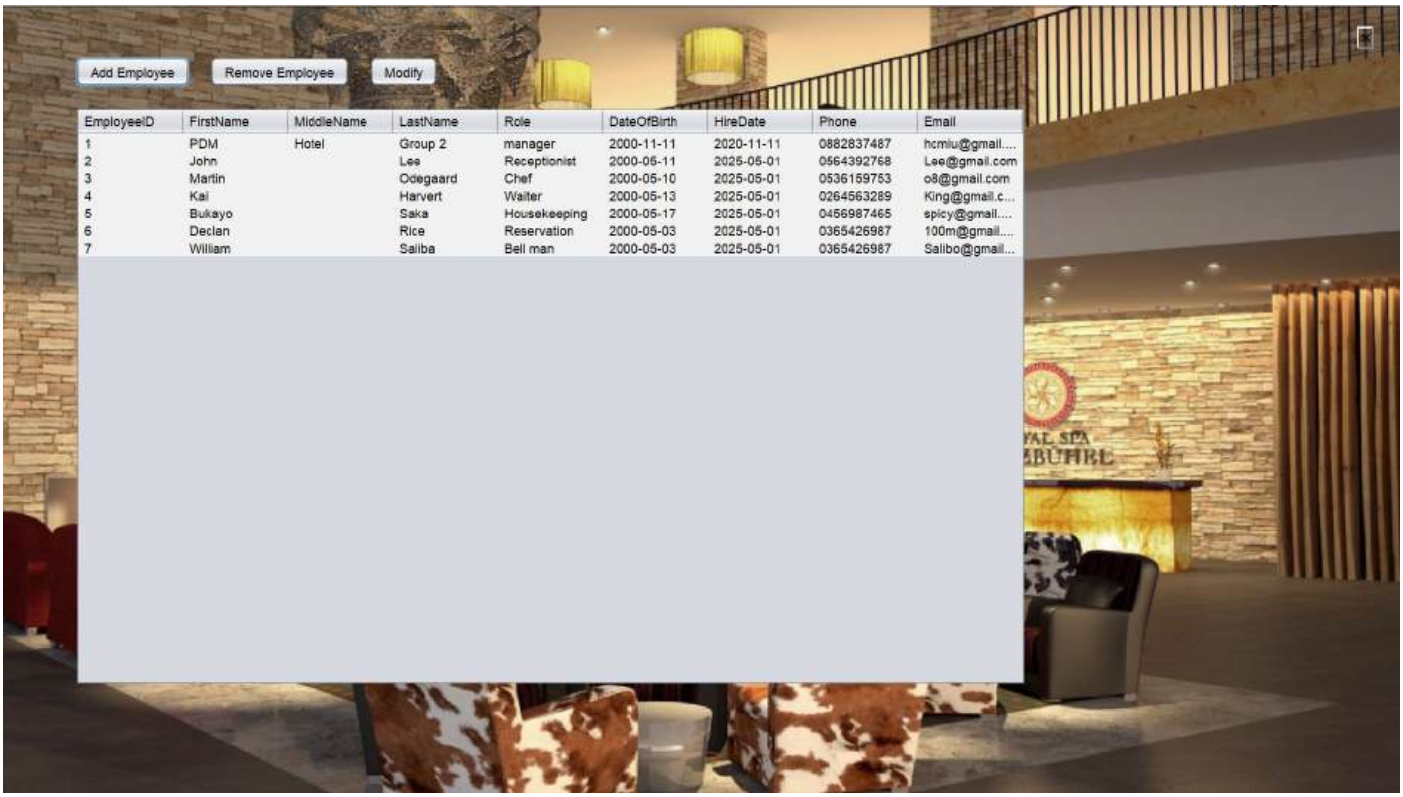


Figure 17: *Dashboard interface*

5.2.3 Manage employee:

The employee table is shown as the image below.



EmployeeID	FirstName	MiddleName	LastName	Role	DateOfBirth	HireDate	Phone	Email
1	PDM	Hotel	Group 2	manager	2000-11-11	2020-11-11	0882837487	hcmiu@gmail....
2	John		Lee	Receptionist	2000-06-11	2025-05-01	0564392768	Lee@gmail.com
3	Martin		Odegaard	Chef	2000-05-10	2025-05-01	0536159753	o8@gmail.com
4	Kai		Harvert	Waiter	2000-05-13	2025-05-01	0264563289	King@gmail.c...
5	Bukayo		Saka	Housekeeping	2000-05-17	2025-05-01	0456987465	spicy@gmail....
6	Declan		Rice	Reservation	2000-05-03	2025-05-01	0365426987	100m@gmail....
7	William		Saliba	Bell man	2000-05-03	2025-05-01	0365426987	Salibo@gmail...

Figure 18: *Employee Management interface*

Adding a new employee:

Figure 19: *Fill the employee information*

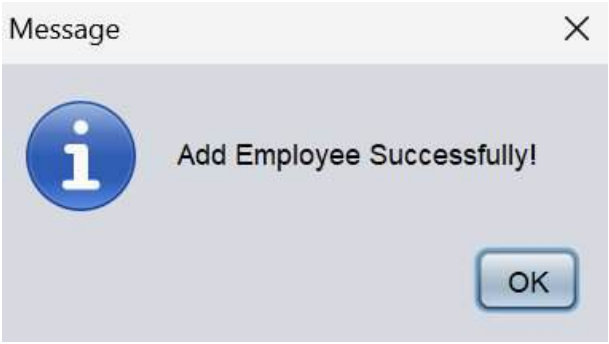


Figure 20: *Success panel show*

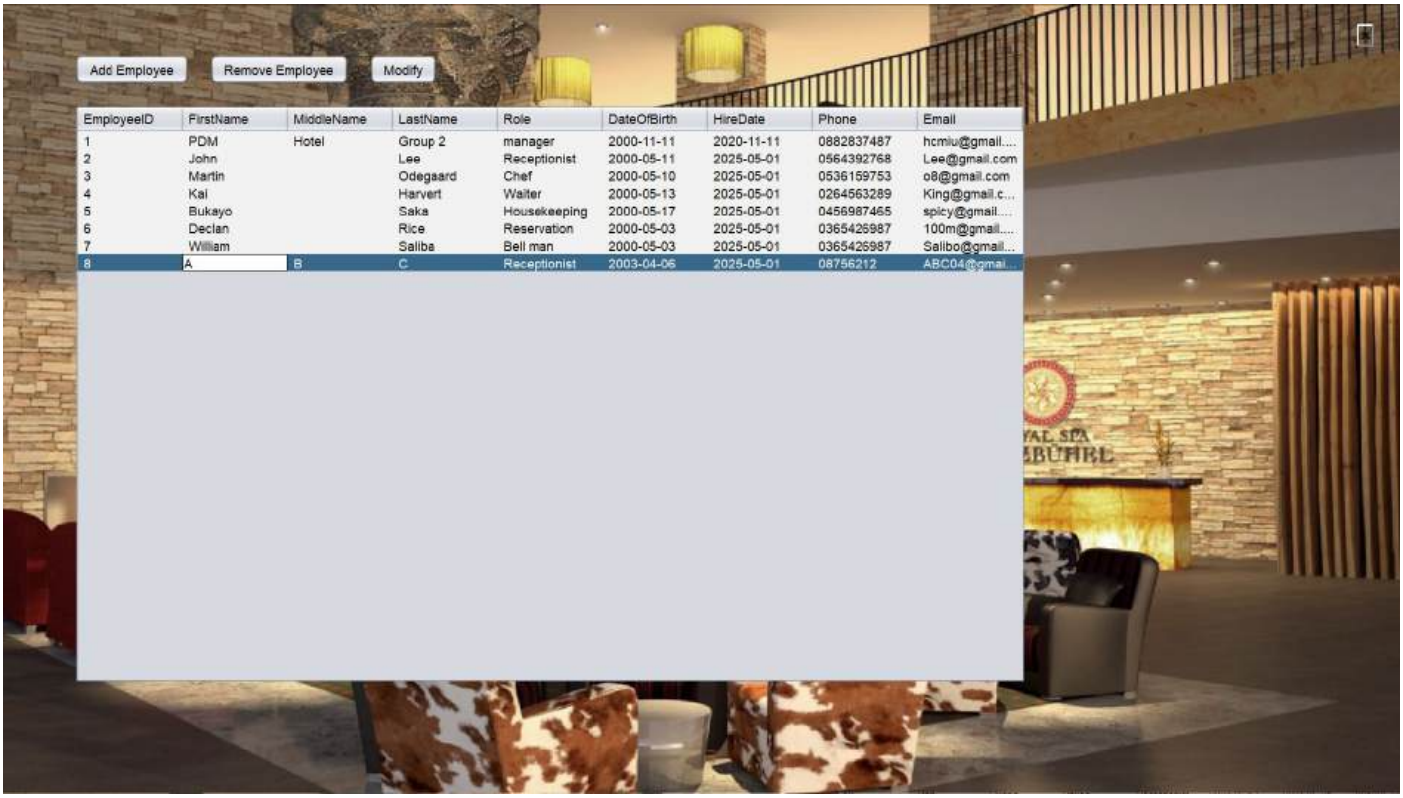


Figure 21: *Choose employee to modify*

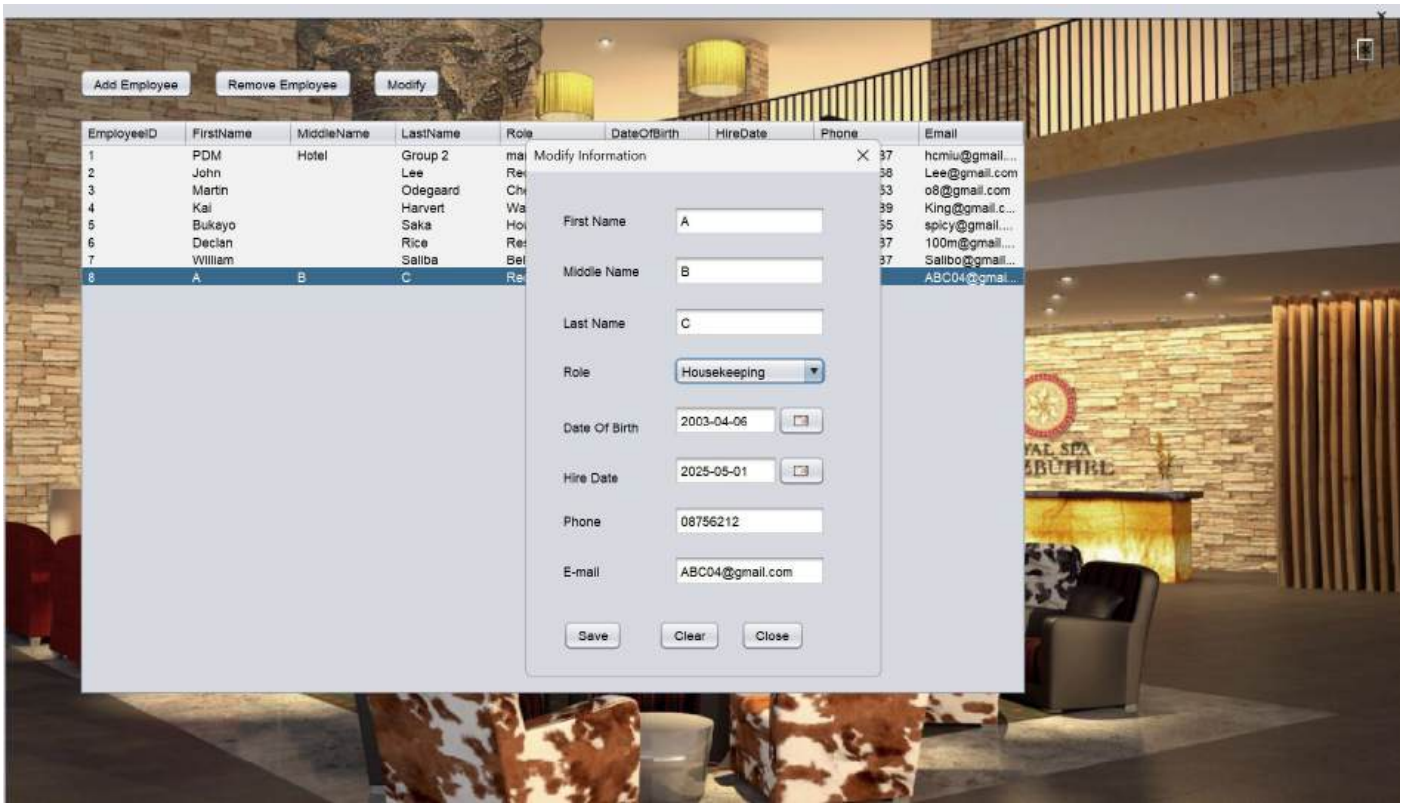


Figure 22: *Modify Information*

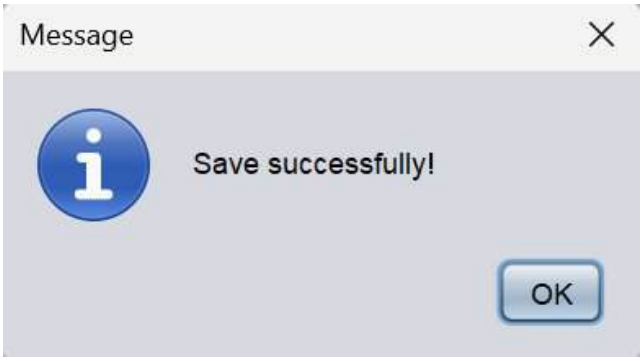


Figure 23: *Success changing panel*

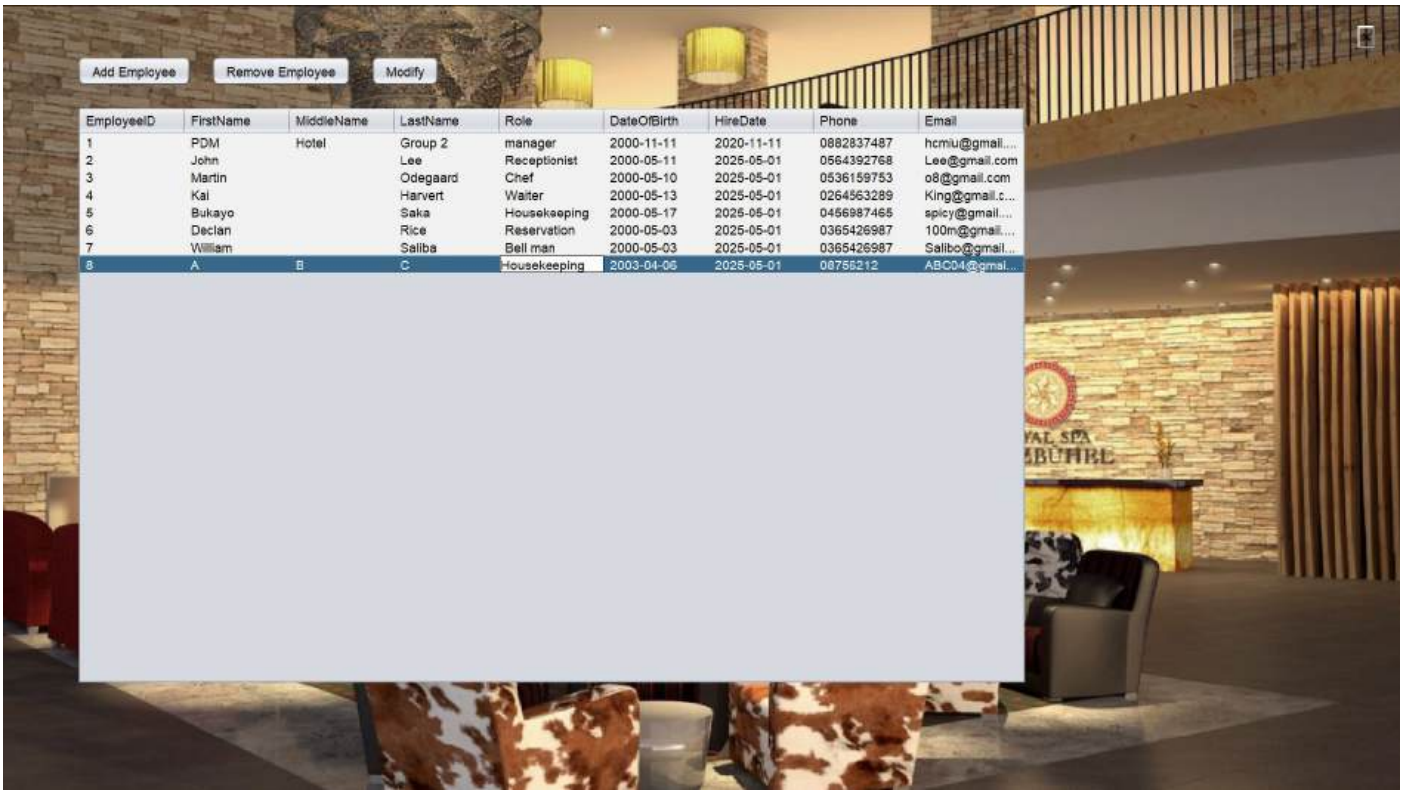


Figure 24: *Value after modification*

Manager chooses the row having EmployeeID = 8, remove the employee will be shown as image below.

Figure 25: *Remove employee*

5.2.4. Manage room:

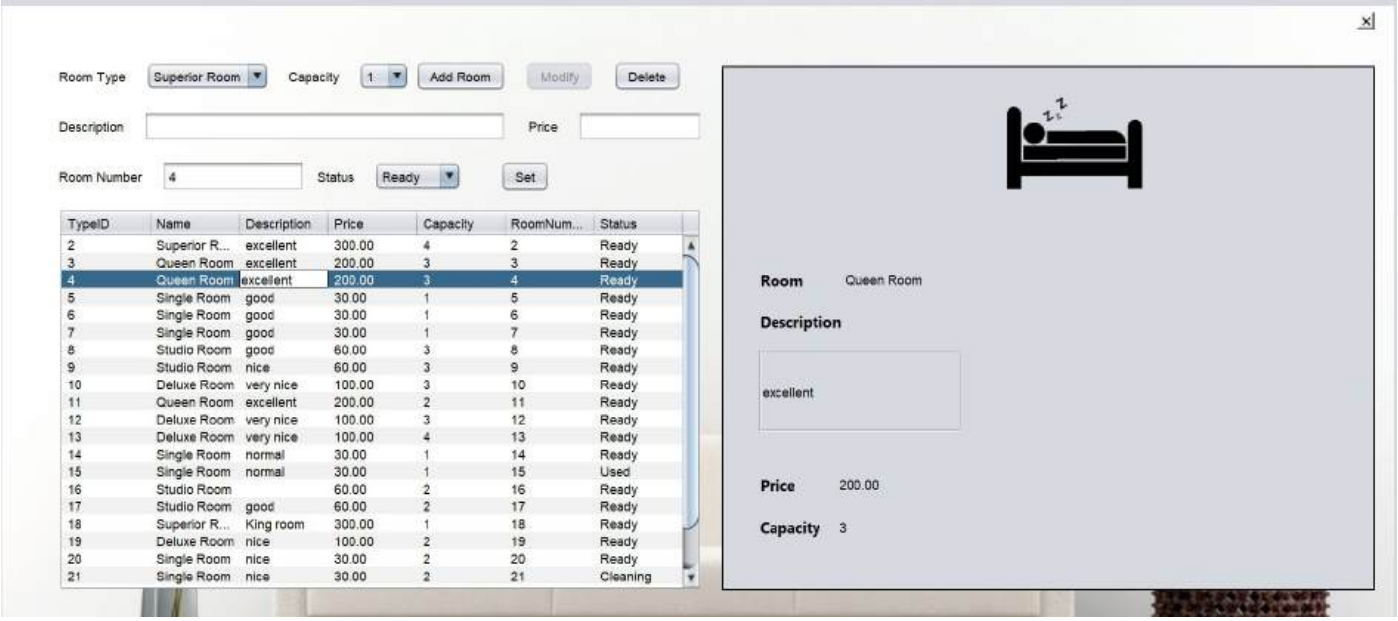


Figure 26: *Room interface*

Manager choose the room type, capacity, price and fill the description textbox to add a new room.

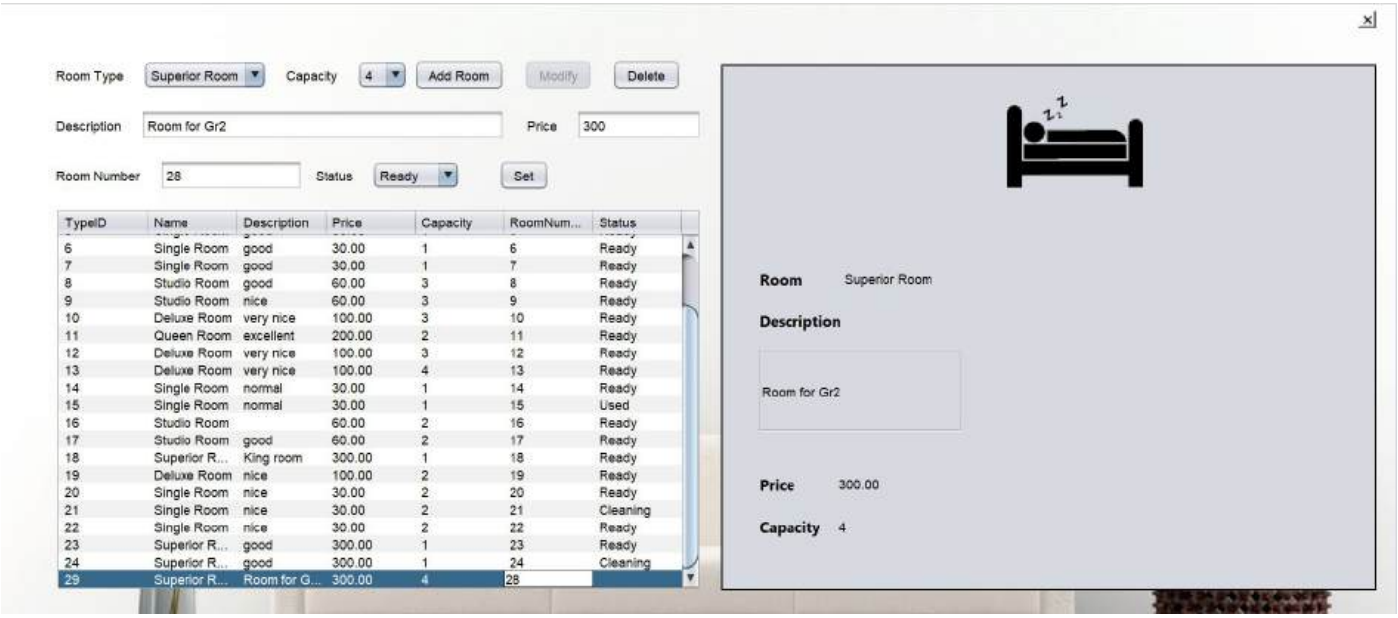


Figure 27: *Add room type information*

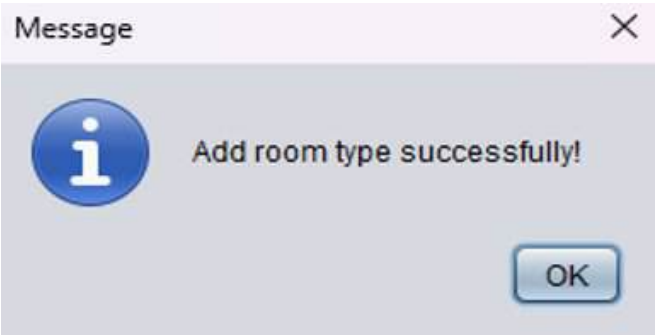


Figure 28: *Adding success*

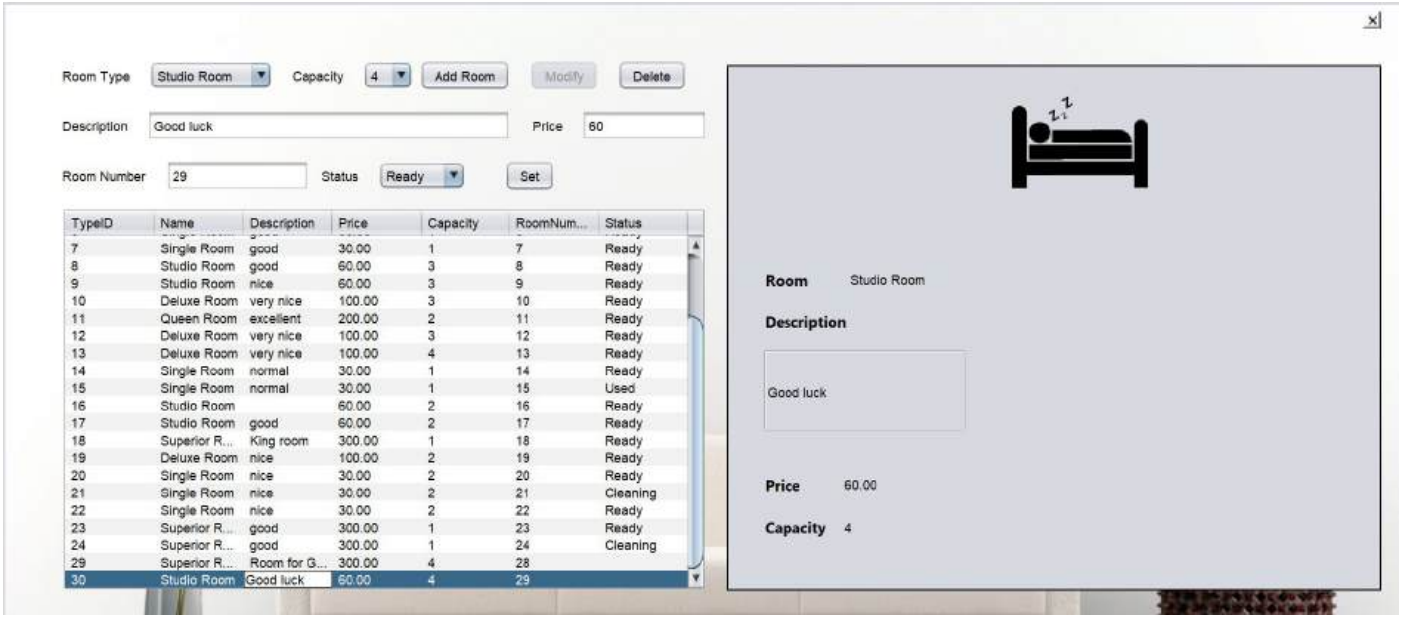


Figure 29: *Room table record a new room*

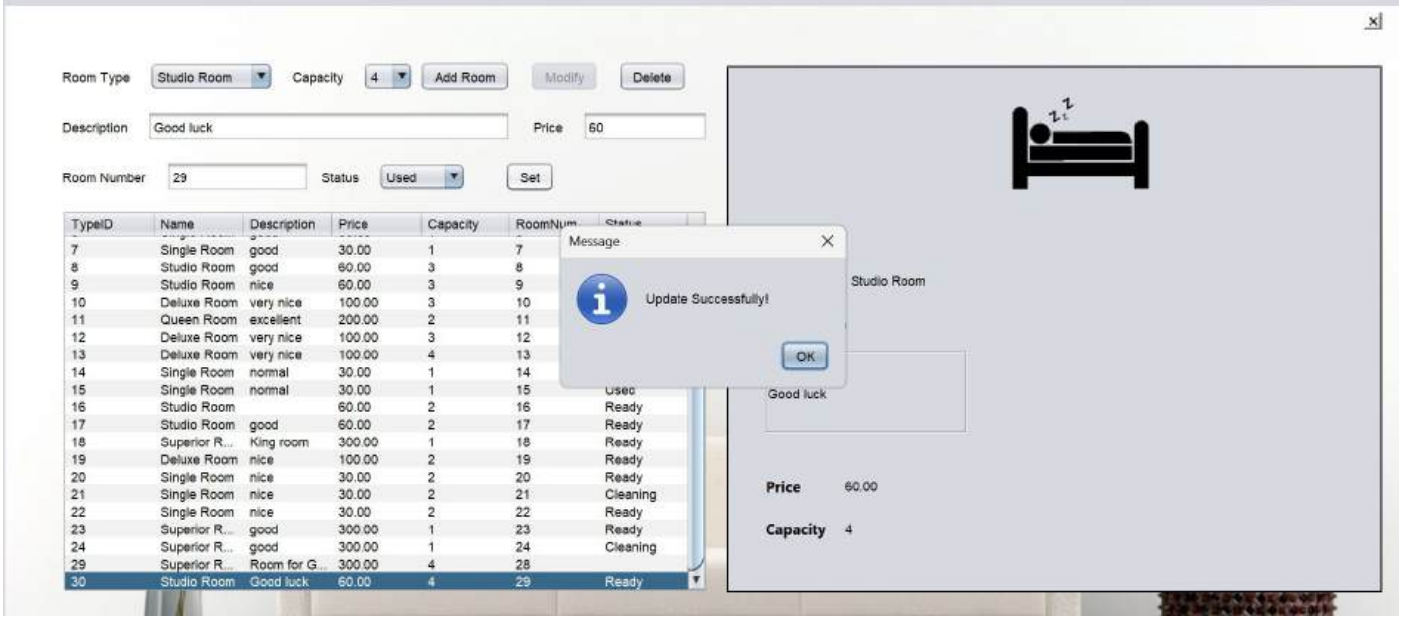


Figure 30: *Set room status successfully*

Figure 31: *Update status*

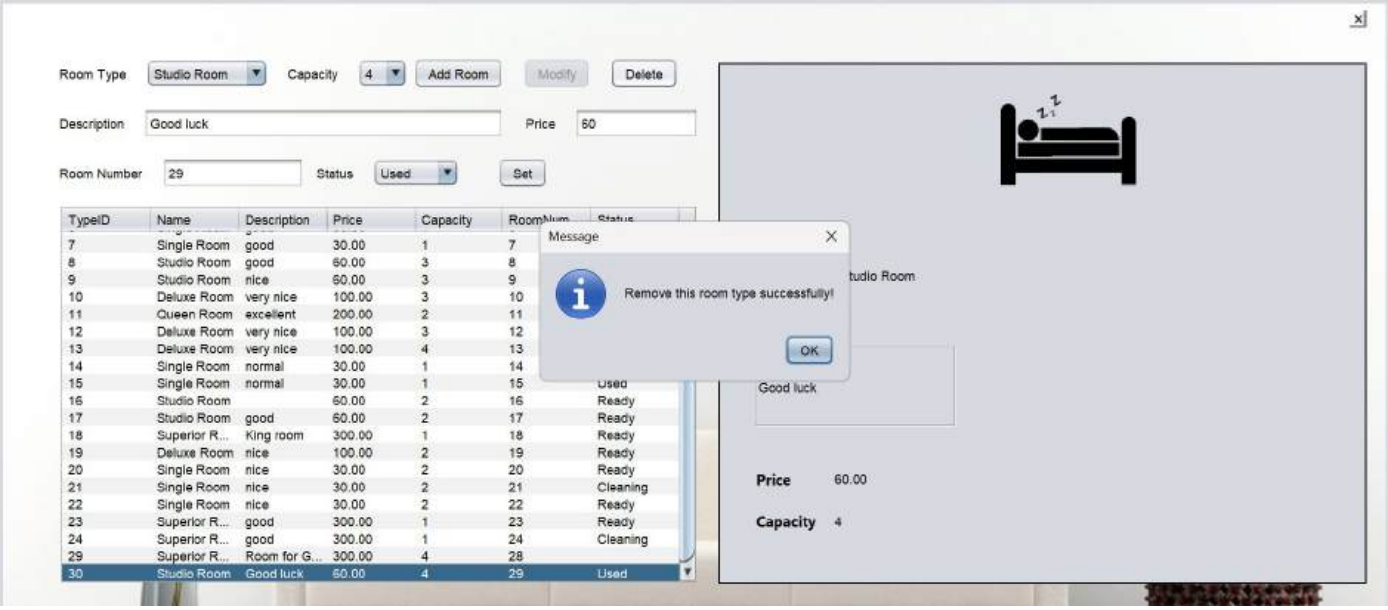


Figure 32: *Remove room announcement*

5.2.5. Manage booking:

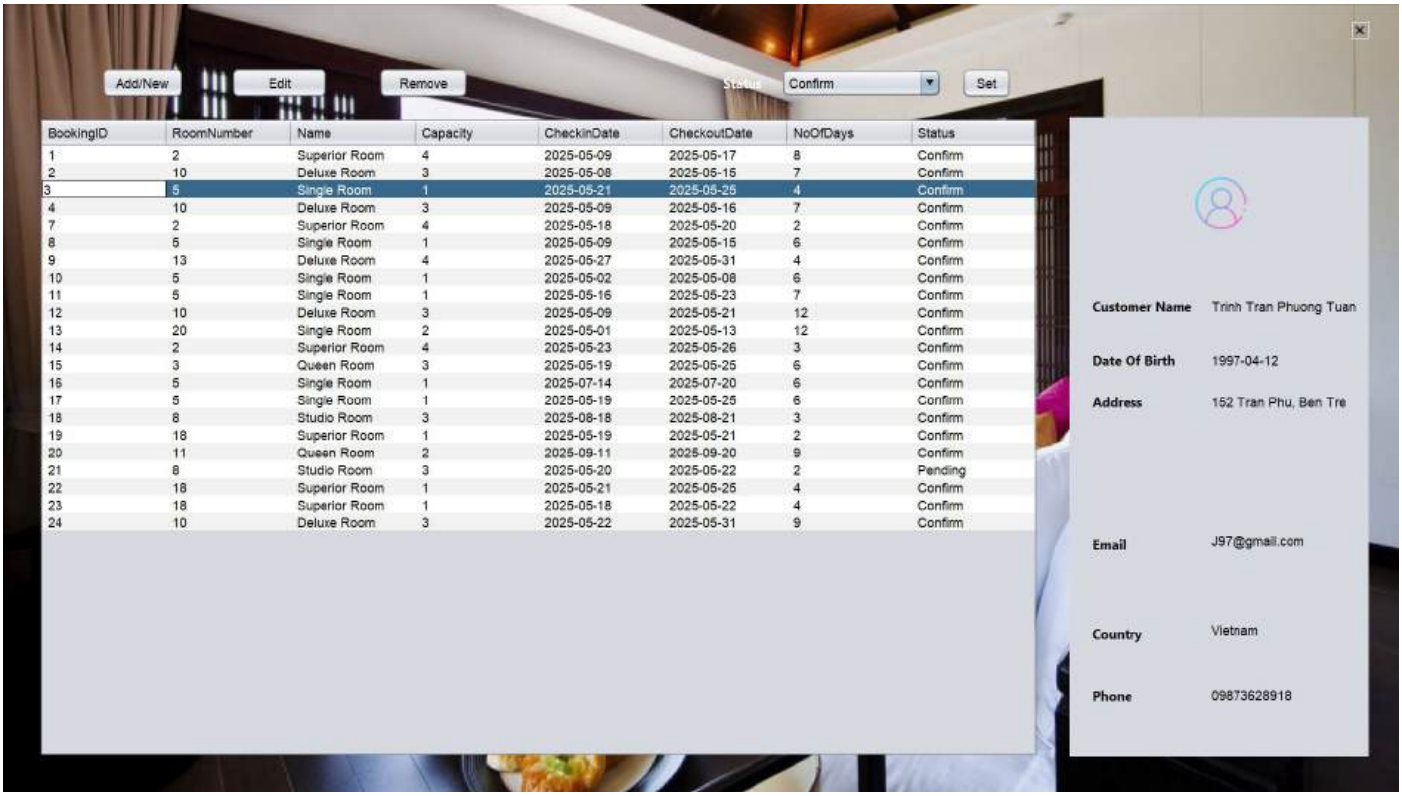


Figure 33: *Booking interface*

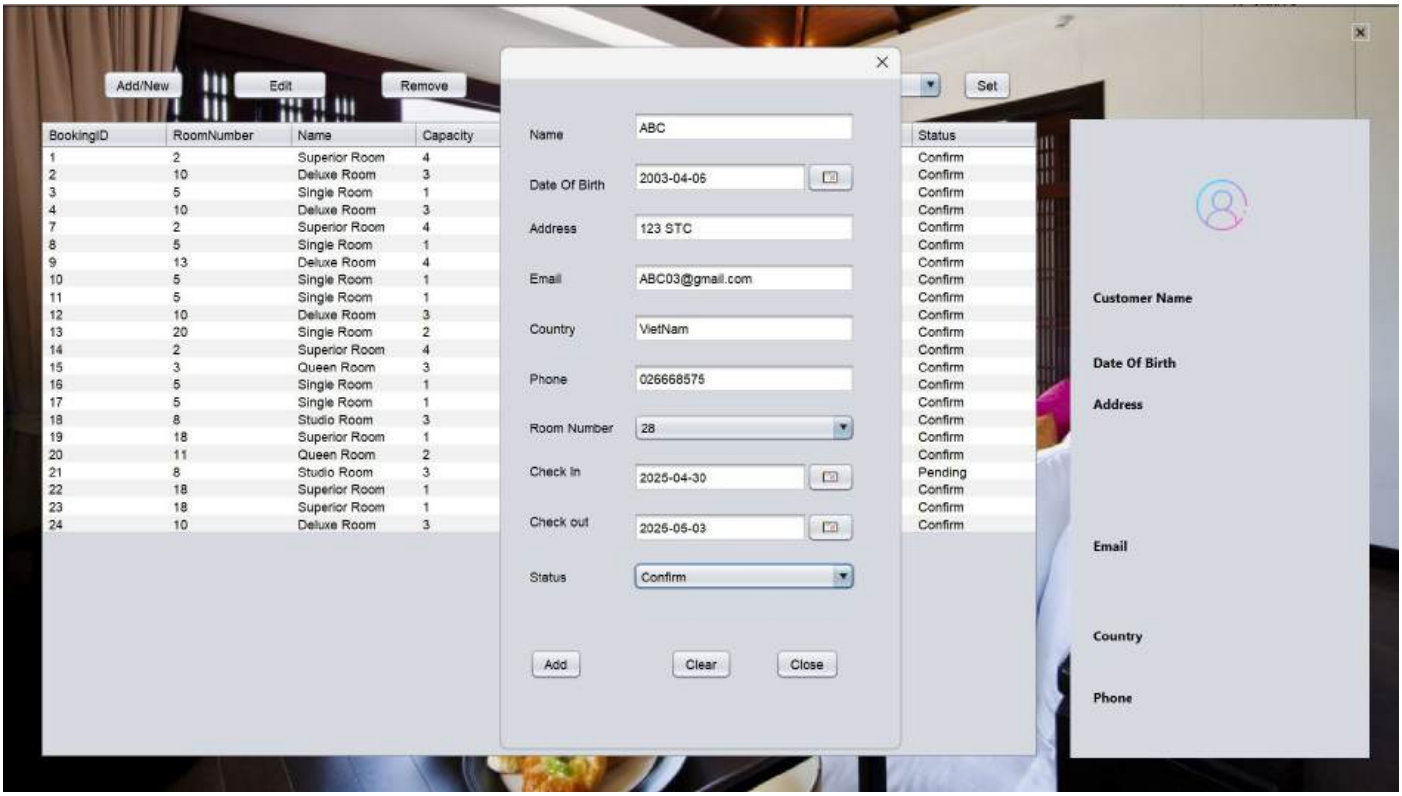


Figure 34: *Filling reservation information*

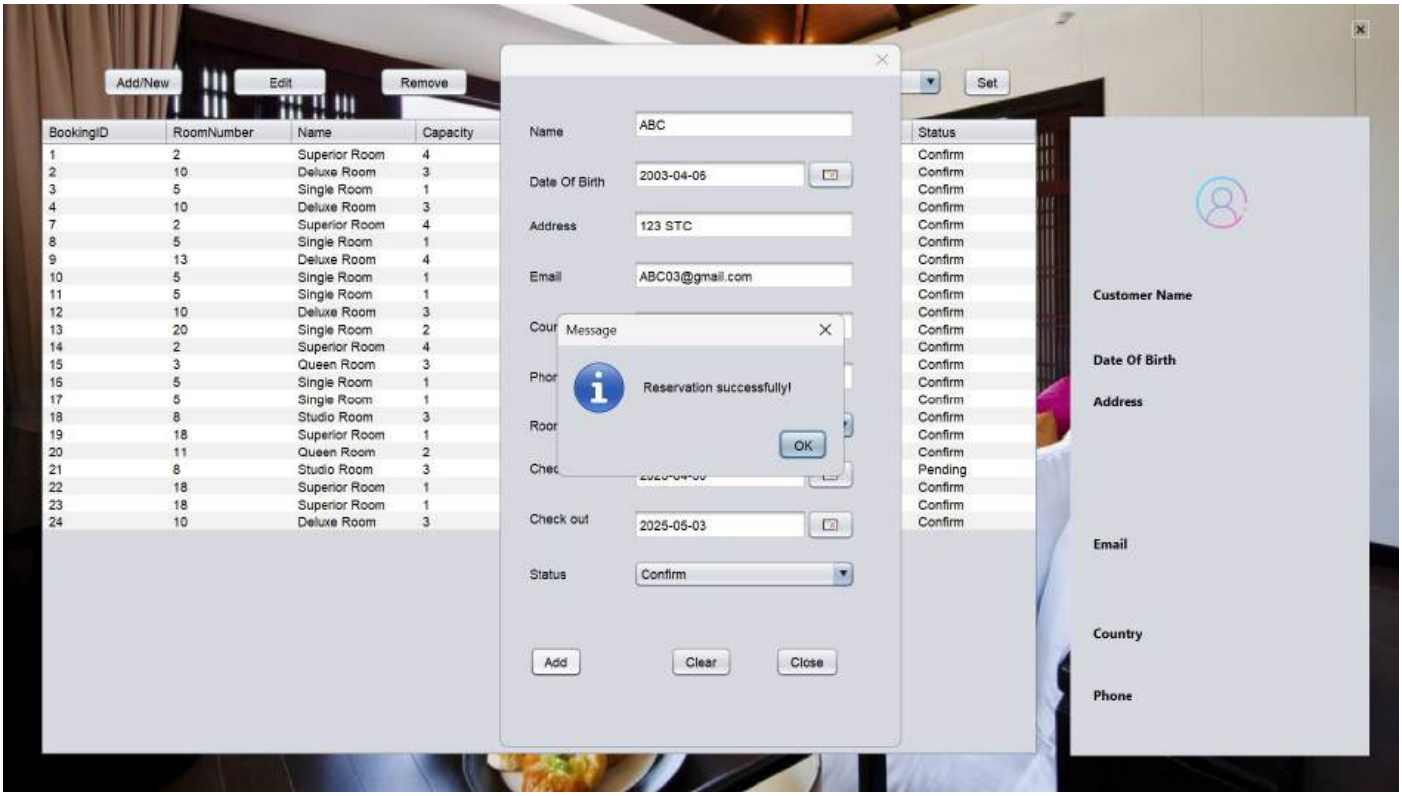


Figure 35: *Reservation successfully*

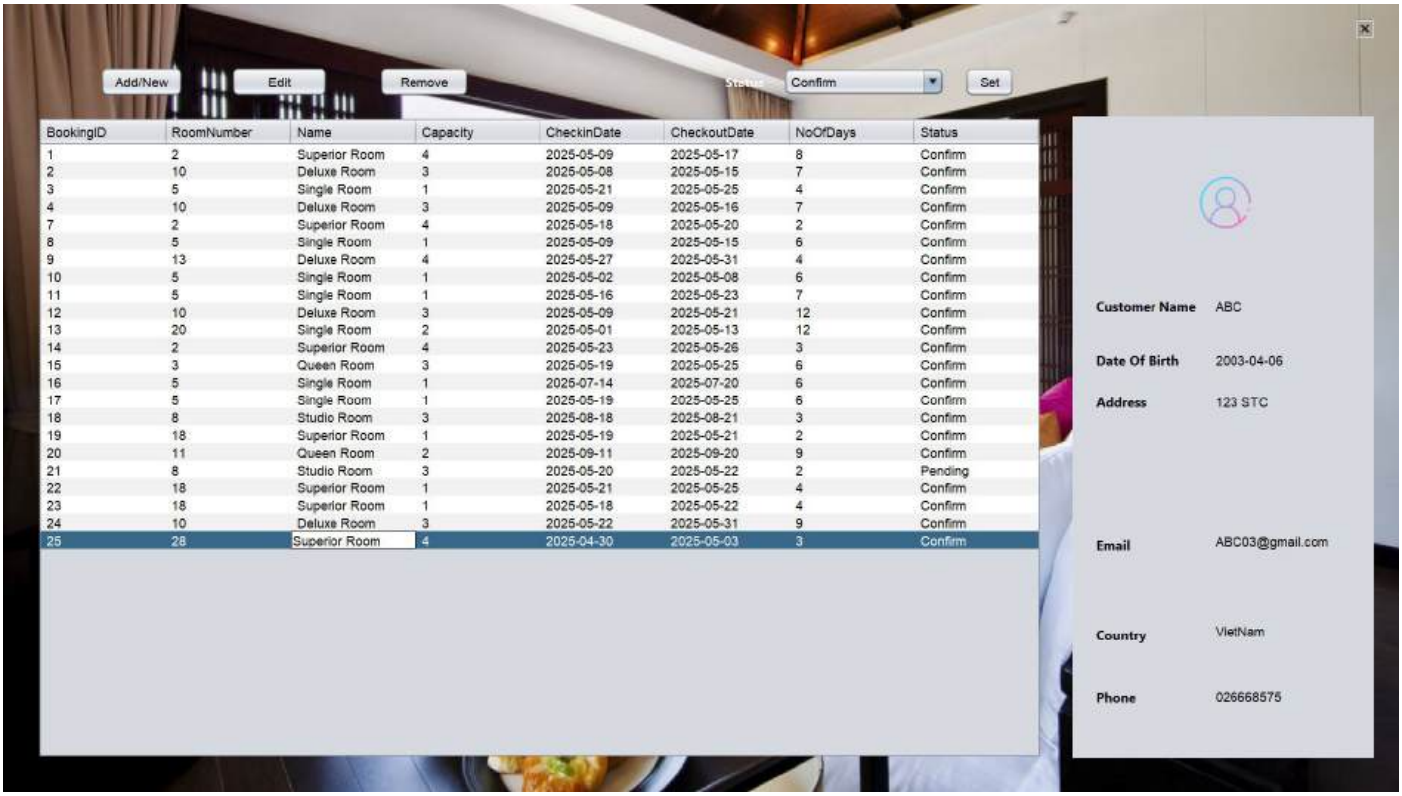


Figure 36: *Save booking information into new table*

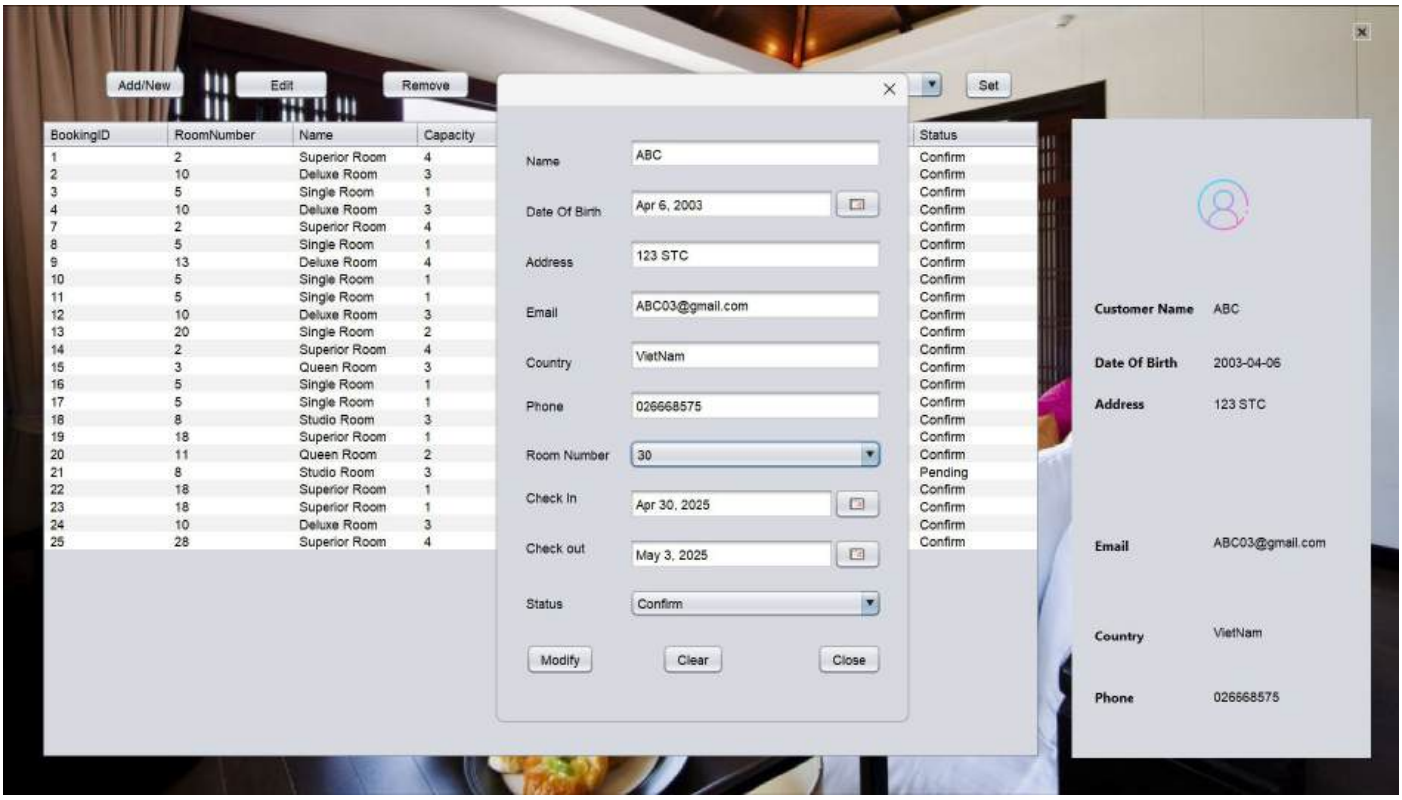
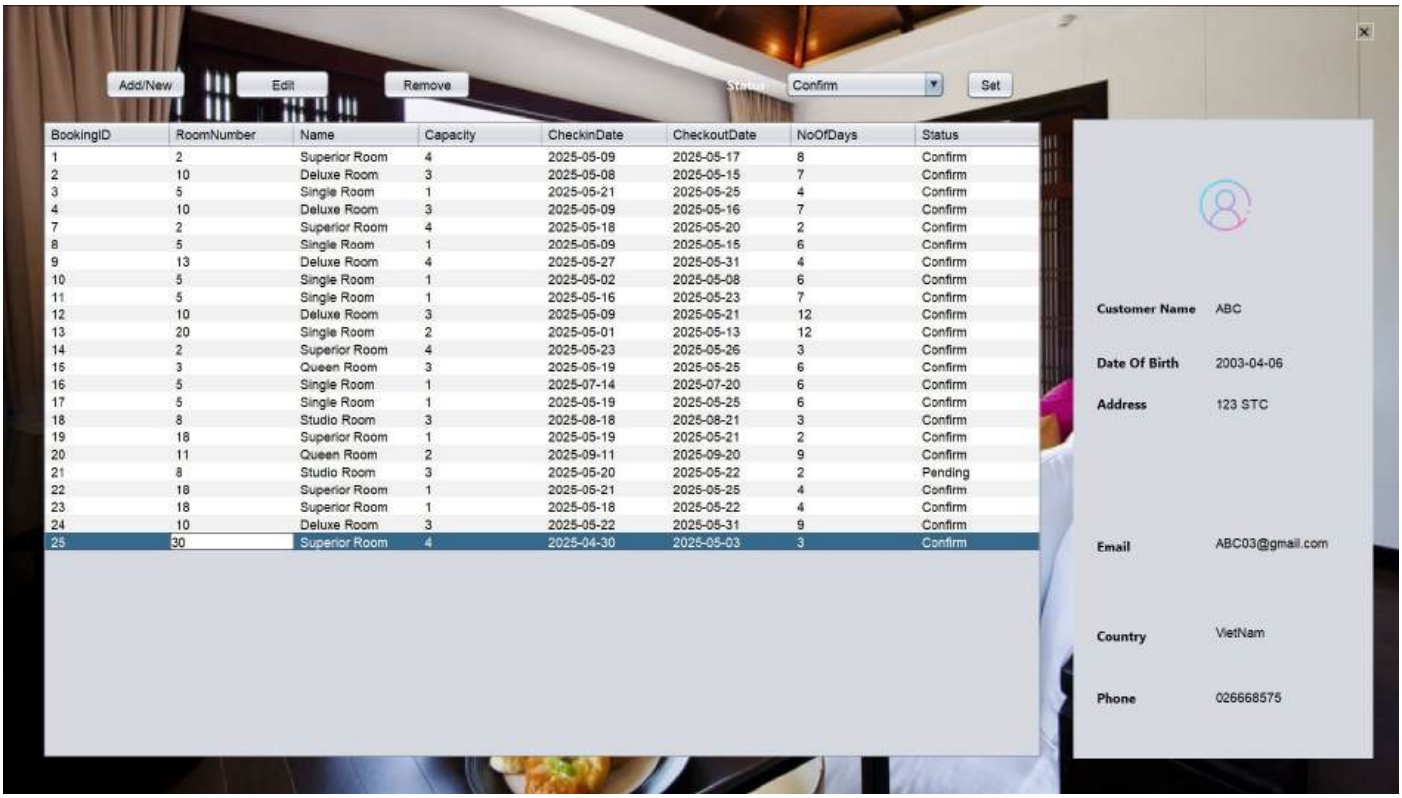


Figure 37: *Modify information*

Figure 38: *Modify successfully*



BookingID	RoomNumber	Name	Capacity	CheckinDate	CheckoutDate	NoOfDays	Status
1	2	Superior Room	4	2025-05-09	2025-05-17	8	Confirm
2	10	Deluxe Room	3	2025-05-08	2025-05-15	7	Confirm
3	5	Single Room	1	2025-05-21	2025-05-25	4	Confirm
4	10	Deluxe Room	3	2025-05-09	2025-05-16	7	Confirm
7	2	Superior Room	4	2025-05-18	2025-05-20	2	Confirm
8	5	Single Room	1	2025-05-09	2025-05-15	6	Confirm
9	13	Deluxe Room	4	2025-05-27	2025-05-31	4	Confirm
10	5	Single Room	1	2025-05-02	2025-05-08	6	Confirm
11	5	Single Room	1	2025-05-16	2025-05-23	7	Confirm
12	10	Deluxe Room	3	2025-05-09	2025-05-21	12	Confirm
13	20	Single Room	2	2025-05-01	2025-05-13	12	Confirm
14	2	Superior Room	4	2025-05-23	2025-05-26	3	Confirm
15	3	Queen Room	3	2025-05-19	2025-05-25	6	Confirm
16	5	Single Room	1	2025-07-14	2025-07-20	6	Confirm
17	5	Single Room	1	2025-05-19	2025-05-25	6	Confirm
18	8	Studio Room	3	2025-08-18	2025-08-21	3	Confirm
19	18	Superior Room	1	2025-05-19	2025-05-21	2	Confirm
20	11	Queen Room	2	2025-09-11	2025-09-20	9	Confirm
21	8	Studio Room	3	2025-05-20	2025-05-22	2	Pending
22	18	Superior Room	1	2025-05-21	2025-05-25	4	Confirm
23	18	Superior Room	1	2025-05-18	2025-05-22	4	Confirm
24	10	Deluxe Room	3	2025-05-22	2025-05-31	9	Confirm
25	30	Superior Room	4	2025-04-30	2025-05-03	3	Confirm

Figure 39: *System receive booking information from website*

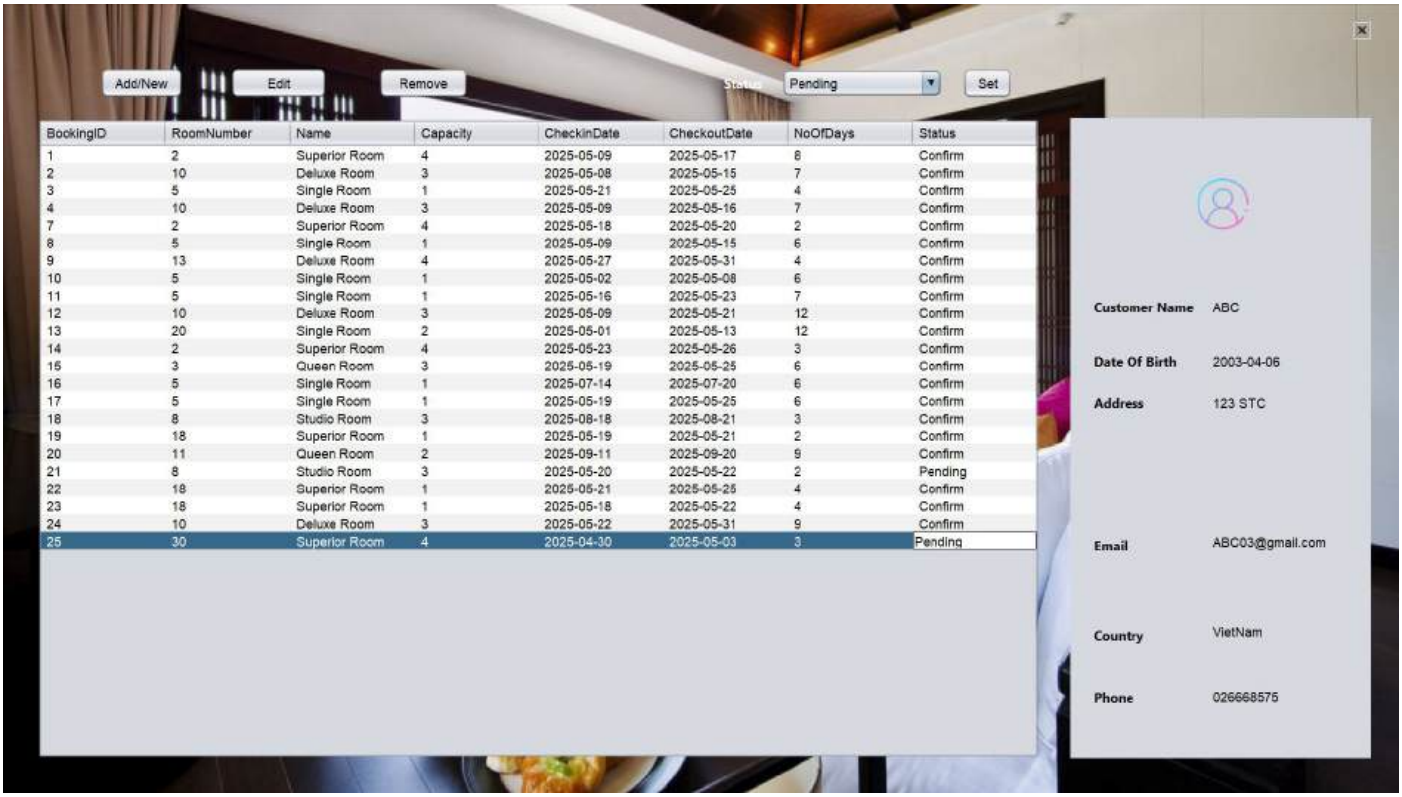


Figure 40: *Update room status*

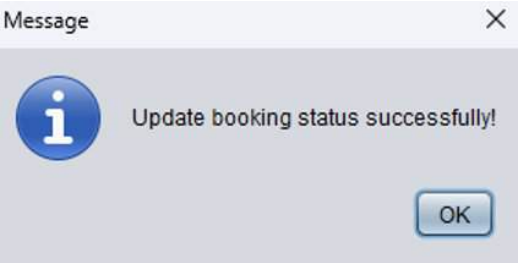


Figure 41: *Update succesfully*

5.2.6. Payment

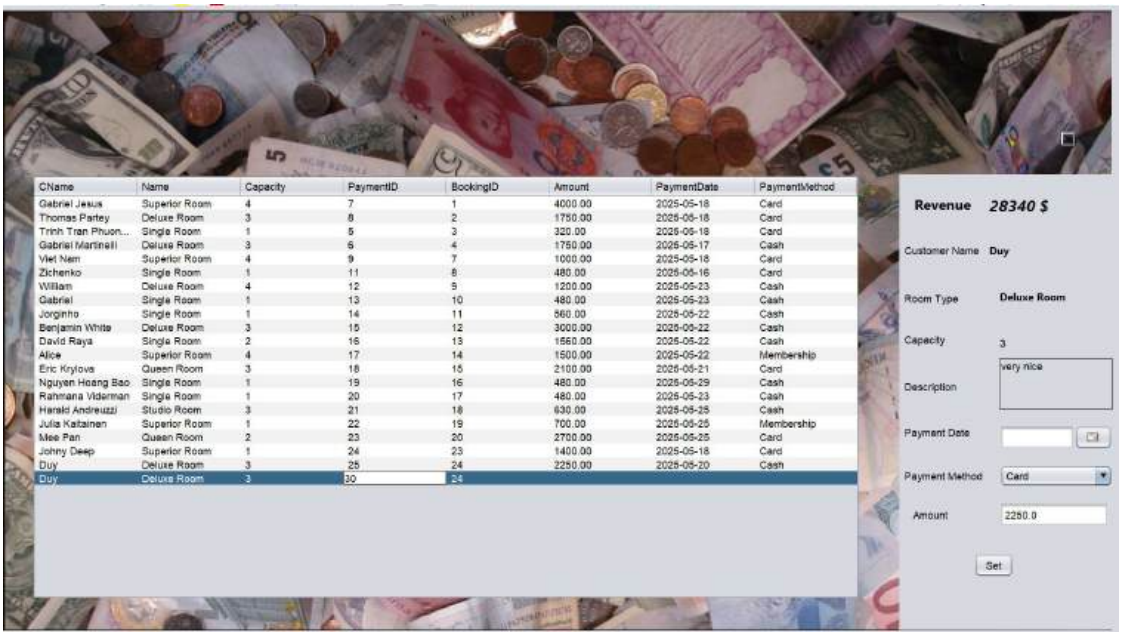


Figure 42: *Payment interface*

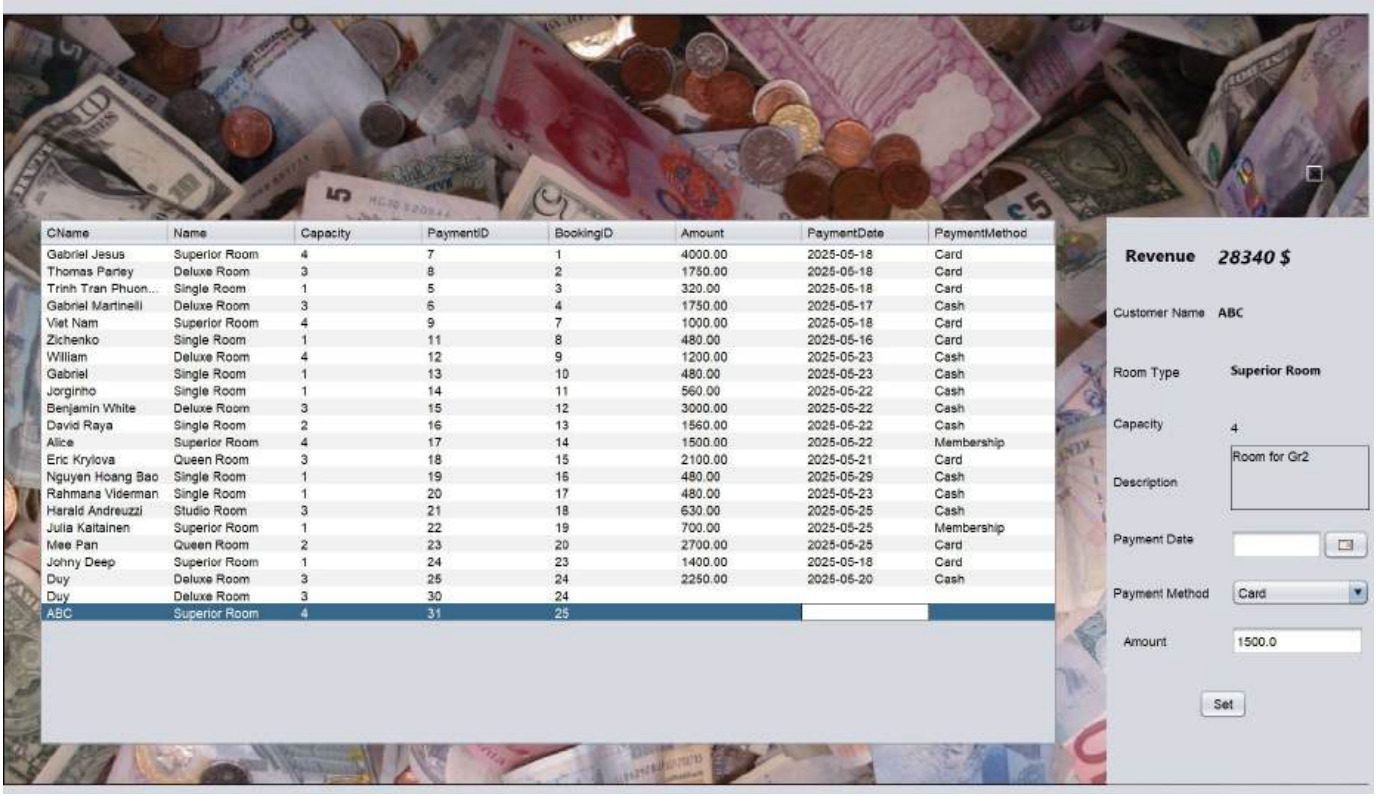


Figure 43: *Update payment details*

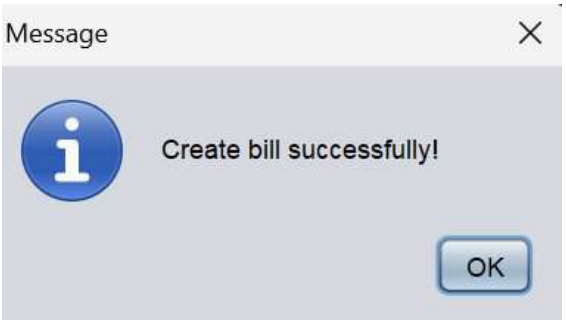


Figure 44: *Update payment successfully*

Figure 45: *Payment information after create bill*

CHAPTER 6. CONCLUSION AND RECOMMENDATION

6.1 . Achieved goals

We are excited to share the successful completion of our hotel management system, meeting all the objectives we set at the start. Here's an overview of our main achievements:

User-Friendly Interface: We focused on creating an intuitive design that ensures easy navigation and a seamless experience for both staff and guests.

Our system provides a complete and efficient solution for managing hotel operations and enhancing guest satisfaction. We remain dedicated to continuous improvement and innovation to maintain our leadership in the hospitality industry.

6.2. Future Work

Although our hotel management system has successfully launched and met its initial objectives, we are committed to ongoing enhancement. Here are several exciting areas we plan to focus on for future development:

- **Enhancing Security:** Protecting data is a top priority. We plan to introduce two-factor authentication to provide an added layer of security for guest information and financial transactions.
- **Adopting New Technologies:** As the hospitality industry continues to evolve, our development team will actively track emerging technologies, including:
    - **Chatbots:** To improve communication by offering 24/7 guest support and addressing frequently asked questions.
- **Staying Ahead of Industry Trends:** We will closely follow industry developments to incorporate features that boost guest satisfaction and operational efficiency, such as:
    - **Mobile Check-In/Check-Out:** Making arrivals and departures faster and more convenient.
    - **Self-Service Kiosks:** Enabling guests to independently manage reservations, request services, or access information.
    - **Enhanced Search Functions:** Improving the search experience for customers to easily find bookings, and adding a search feature for employees to quickly access information across all system functions.

By focusing on these advancements, we are confident our hotel management system will continue to be a powerful tool, maintaining a strong competitive advantage in the rapidly evolving hospitality industry.

6.3. Concluding thought

In conclusion, we believe our hotel management system has strong potential for future growth. We are committed to continually enhancing its efficiency and ease of use for both staff and guests. This project has been an invaluable learning experience, helping our team strengthen essential skills in collaboration, communication, and problem-solving — all of which will benefit us in future projects.

We sincerely thank everyone who contributed to the development of this system; their support and expertise were vital to our success. Looking ahead, we are dedicated to making a positive impact in the hospitality industry by improving operations and elevating the guest experience. With a focus on ongoing innovation and development, we are excited about the opportunities our hotel management system offers to transform hotel operations.

REFERENCES

Get user interface, Github Tushar Pankhaniya: tushar-2223/Hotel-Management-System: A PHP + MySQL + HTML + CSS + JS based Hotel Management System with a beautiful user interface (github.com) (1.5.2025)

Hotel Management System in java | BTech Days: https://www.youtube.com/watch?v=1P77ck8lqH8&list=PLdRq0mbeEBmzHDbDWyWuu6h0DoFNvDyoi&pp=iAQB (7.5.2025)

1NF, 2NF, 3NF and BCNF in Database Normalization | freeCodeCamp.org Database Normalization – Normal Forms 1nf 2nf 3nf Table Examples (freecodecamp.org) (1.5.2025)

Install PHP library | AZDIGI: Hướng Dẫn Cài đặt LAMP Stack Trên Ubuntu 22.04 (azdigi.com) (1.5.2025)

Setup PHP and XAMPP | freeCodeCamp : https://www.freecodecamp.org/news/how-to-get-started-with-php/ (1.5.2025)

Allow remote MySQL | Digitalocean: How To Allow Remote Access to MySQL | DigitalOcean (1.5.2025)

Convert date to JDate | JCalendar: JCalendar – toedter.com (23.5.2025)